

GPU 101

How a machine for pictures learned to compute $A + B$

Minjae Gwon
POSTECH
September 10, 2026

그림을 그리던 기계는 어떻게 A와 B를 더하는 계산기가 됐을까요? 오늘은 이 질문에서 출발해, GPU가 왜 지금과 같은 구조를 갖게 됐는지 이야기해보겠습니다.

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

3. The execution engine

- Warps and SIMT
- SMs and GPU organization

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Results and completion

Contents

발표는 네 부분입니다. GPU를 이해하려는 동기에서 시작해 그래픽의 요구가 셰이더와 CUDA, 하드웨어를 함께 바꾼 과정을 보겠습니다. 이어 실행 엔진과 AI 시대의 변화를 살펴보고, 마지막으로는 CUDA 생태계를 거쳐 PyTorch의 덧셈 한 줄이 처리되는 과정을 따라가겠습니다.

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

3. The execution engine

4. Following A + B

Motivation

먼저 GPU를 쓰면서 마주치는 궁금증부터 짚어보겠습니다.

What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

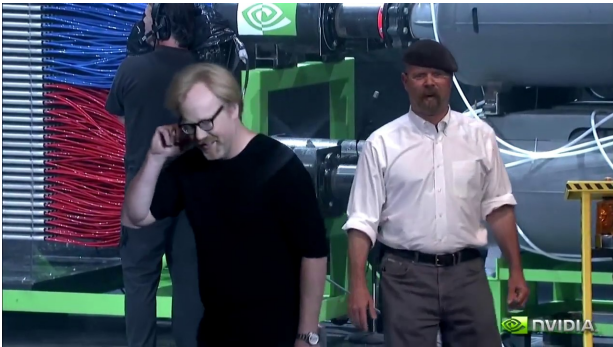
[Pictures](#) → [Execution engines](#) → [C = A + B](#)

What should make more sense after this talk?

GPU를 쓰다 보면 nvidia-smi를 켜서 메모리 사용량을 보거나, 모델을 올렸는데 왜 기다리는지 궁금해지는 순간이 있습니다. PyTorch나 Hugging Face에서는 몇 줄로 끝나는 작업이 아래에서는 어떻게 실행되는지도 잘 보이지 않습니다.

오늘은 VRAM에 데이터를 올리는 것과 GPU가 계산하는 것이 어떻게 연결되는지, 프레임워크의 요청 아래에서 무슨 일이 일어나는지 이해할 기본 그림을 만들겠습니다. 그래픽과 AI가 요구하는 계산이 바뀔 때 하드웨어와 소프트웨어가 어떻게 함께 발전했는지도 연결해보겠습니다. 먼저 많은 일을 함께 처리한다는 것이 어떤 모습인지 시연으로 보겠습니다.

One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

CPU vs. GPU

One paintball at a time.

Many paintballs together.

▶ [Watch the demo](#)

One image. Very different ways to paint it.

[진행 메모: 시간이 허용되면 사진이나 오른쪽 링크를 눌러 영상을 재생한다. 아래는 읽을 대사.]

MythBusters가 2008년 NVIDIA 행사에서 보여준 시연입니다. 한쪽은 페인트볼을 하나씩 쏘고, 다른 쪽은 많은 페인트볼을 함께 쏘서 그림을 만듭니다. 같은 그림을 완성하는 두 방식의 차이에 주목해보 시면 됩니다.

실제 CPU도 병렬로 계산할 수 있고, 어느 방식이 빠른지는 작업에 따라 달라집니다. 이 시연이 잘 보여주는 것은 여러 위치의 작업을 함께 진행하는 모습입니다. 그림을 그리는 일에는 왜 이런 병렬성이 많았을까요?

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

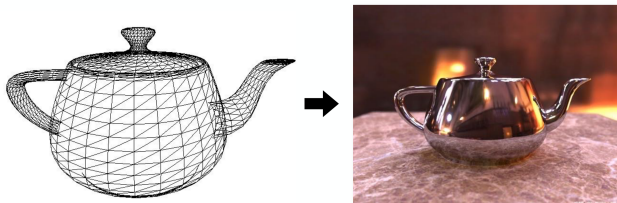
- Rendering and shaders
- GPGPU and CUDA

From pictures to programs

그래픽이 원하는 계산을 어떻게 바꿨고, 그 요구에 API와 하드웨어가 어떻게 함께 발전했는지 보겠습니다.

The original deadline: draw the next frame

3D rendering



Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Image credit: Henrik Wann Jensen

Geometry + materials + lights + camera → an image

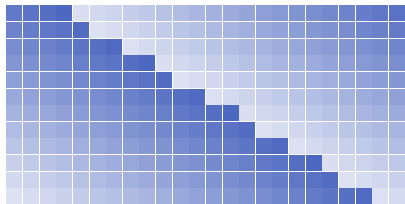
The original deadline: draw the next frame

게임 속에서 햇빛이 비치는 호숫가를 걷는다고 생각해봅시다. 카메라가 움직이면 화면에 보이는 물결과 가려짐, 반사 모습이 달라집니다. 컴퓨터는 물체의 삼각형들, 재질, 조명, 카메라 위치를 가지고 다음 화면을 계산해야 합니다. 이 작업이 렌더링입니다.

화면의 주전자도 마찬가지입니다. 왼쪽에는 표면의 모양을 나타내는 기하 정보가 있고, 오른쪽에는 빛과 재질을 반영한 이미지가 있습니다. GPU는 원래 이런 3차원 그래픽을 빠르게 처리하기 위해 발전했습니다. 게임에서는 이 계산을 반복해 다음 장도 정해진 시간 안에 계속 만들어야 합니다.

Stanford CS149, Fall 2025 · slide 5 · image: Henrik Wann Jensen

Millions of positions. The same kind of calculation.



Different positions, different inputs

 1920×1080

output pixels per frame

 $\times 60 \text{ frames / second}$ $\approx 124 \text{ million}$

output pixels per second

The opportunity is repeated work across independent data.

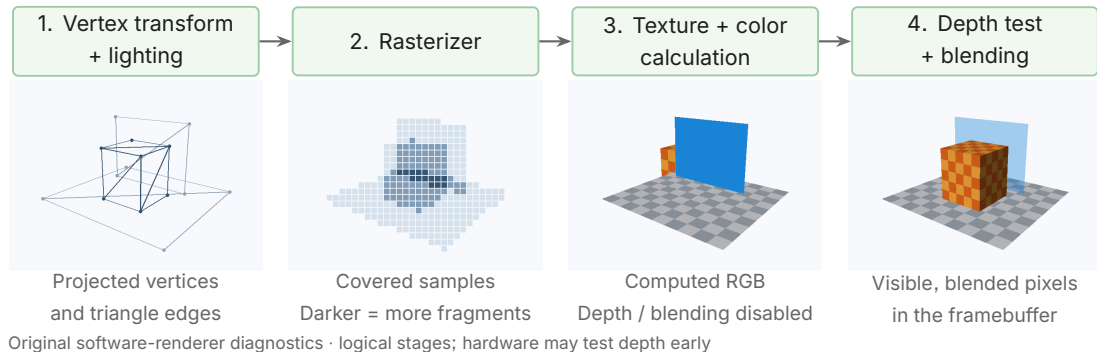
Millions of positions. The same kind of calculation.

1920 곱하기 1080 화면에는 약 200만 개의 픽셀이 있습니다. 이것을 초당 60장 만들면 매초 약 1억 2천4백만 개의 출력 픽셀을 만들어야 합니다. 이것은 연산 횟수가 아니라 출력 픽셀 수입니다. 하나의 픽셀을 정하기 위해서는 훨씬 많은 계산이 필요할 수 있습니다.

여기서 중요한 것은 단순히 숫자가 크다는 사실만이 아닙니다. 수면의 이 지점에서도 빛의 반사를 계산하고, 바로 옆 지점에서도 비슷한 계산을 합니다. 위치와 방향 같은 입력은 다르지만 적용할 계산법은 같을 수 있습니다. 서로 다른 위치에 대해 비슷한 계산을 아주 많이 한다는 성질이 GPU가 활용할 수 있는 병렬성의 출발점입니다.

A scene becomes pixels, one stage at a time

SAME MESH · SAME CAMERA · SAME DRAW ORDER



Vertices → **fragments** → **fragment colors** → **framebuffer pixels**.

Pipeline reference: NVIDIA · The Cg Tutorial, Ch. 1 · original scene and diagnostic renders

A scene becomes pixels, one stage at a time

같은 카메라로 큐브·바닥·반투명 파란 판을 봅니다. 중간 단계는 완성된 그림이 아니므로, 자체 소프트웨어 렌더러의 데이터를 눈에 보이게 표시했습니다.

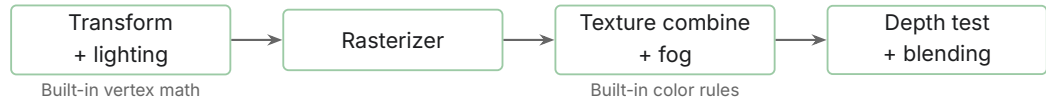
첫 그림은 변환된 정점과 삼각형의 연결입니다. 선은 설명용 표시이며 GPU가 선 그림을 먼저 저장한다는 뜻은 아닙니다. Rasterizer는 삼각형이 덮는 샘플 위치에 프래그먼트를 만듭니다. 두 번째의 칸은 크게 표시했고, 진한 칸에는 여러 삼각형의 후보가 겹쳐 있습니다.

세 번째는 텍스처와 조명으로 계산한 색입니다. 깊이 검사와 혼합을 꺼서, 나중에 그린 판과 바닥이 앞의 큐브까지 덮습니다. 네 번째는 깊이 비교와 투명도 혼합을 적용한 결과입니다. 큐브가 앞에 보이고 파란 판 너머로 바닥이 비칩니다.

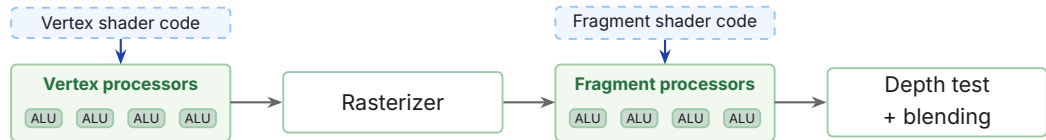
이것은 논리적 순서이며 실제 GPU는 깊이 검사를 일찍 하기도 합니다. 이제 정점과 표면색의 공식을 직접 바꾸고 싶다면 하드웨어가 어떻게 달라져야 할지 보겠습니다.

Richer materials needed programmable hardware

EARLY OPENGL · SETTINGS FOR FIXED-FUNCTION HARDWARE



EARLY PROGRAMMABLE GPU · APPLICATION SUPPLIES PROGRAMS



Pipeline order, not a floorplan · texture sampling hardware remains · output goes to the framebuffer

Custom material formulas need both code and processors that can execute it.

NVIDIA · The Cg Tutorial, Chapter 1 · simplified hardware stages and illustrative unit counts

Richer materials needed programmable hardware

방금 본 네 단계를 같은 위치에 놓고, 하드웨어가 어떻게 바뀌었는지 보겠습니다. 윗줄은 고정 기능 GPU의 주요 처리 순서입니다. 정점의 위치와 조명을 계산하고, rasterizer가 삼각형을 화면의 조각으로 펼칩니다. 텍스처를 조합하고 안개 효과를 적용한 뒤, 깊이 검사와 색 혼합을 거쳐 프레임버퍼에 기록합니다. 초기 OpenGL은 조명과 재질 같은 설정으로 준비된 연산을 제어했습니다.

수면의 물결과 반사를 원하는 방식으로 계산하려면 이 공식 자체를 바꾸고 싶어집니다. 아랫줄은 같은 단계에 프로그램을 실행하는 프로세서가 들어온 모습입니다. 파란 정점·프래그먼트 코드가 초록 실행부에 명령으로 들어갑니다. ALU는 실제 산술 계산을 하는 회로이고, 칸 수는 예시입니다.

가운데 rasterizer와 마지막 검사·혼합은 여전히 전용 하드웨어입니다. 텍스처를 읽는 장치도 남아 있습니다. 전체 파이프라인을 소프트웨어로 바꾼 것이 아니라, 일부 계산 단계에 명령을 실행할 하드웨어를 갖추고 API가 그 프로그램을 제공하게 된 것입니다. 이제 프래그먼트 단계 안으로 들어가 보겠습니다.

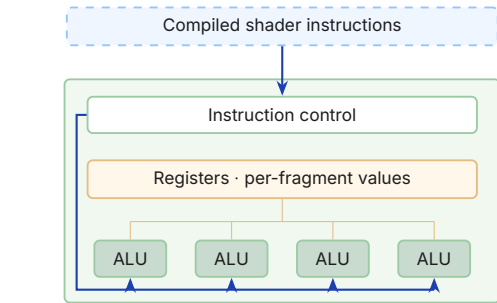
Shader code becomes instructions for physical processors

SOFTWARE · ONE FRAGMENT'S WATER SHADING

```
normal = ripple_normal(uv, time)
reflected = sample_reflection(view, normal)
weight = reflection_weight(view, normal)
output = mix(water_color, reflected, weight)
```

- The program chooses the calculation.
- Each fragment supplies different inputs.

Code supplies instructions; registers hold values; ALUs perform the arithmetic.



The same units process many fragments.

NVIDIA · The Cg Tutorial, Chapter 1 · original pseudocode and selected-resource schematic

Shader code becomes instructions for physical processors

셰이더는 그래픽 처리의 한 단계에서 실행되는 프로그램입니다. 삼각형을 화면에 펼친 조각인 프래그먼트마다 입력을 받아 색을 계산합니다. 이 조각은 가려짐 검사 등을 거쳐 최종 픽셀에 기여할 수 있는 후보입니다.

왼쪽은 물결 방향을 구하고 반사된 풍경을 읽어, 보는 각도에 따라 물의 색과 섞는 의사 코드입니다. 이 프로그램을 컴파일한 명령이 오른쪽 제어부로 들어갑니다. 제어부는 명령을 실행시키고, 레지스터에는 각 프래그먼트의 입력과 중간값이 보관됩니다. 실제 곱셈과 덧셈은 아래의 여러 ALU가 수행합니다. 텍스처 읽기 장치 등의 세부는 생략했습니다.

파란색은 프로그램, 초록색은 실행 하드웨어, 주황색은 값의 저장 공간과 이동입니다. 프래그먼트마다 ALU를 하나씩 새로 만드는 것이 아니라 같은 실행 자원이 많은 프래그먼트를 처리합니다. 이제 GPU가 우리가 쓴 명령을 많은 데이터에 적용할 수 있다면, 그 숫자가 꼭 그림을 위한 값이어야 할까요?

Do those numbers have to be colors?

THE GRAPHICS INTERPRETATION



THE COMPUTE INTERPRETATION



GPGPU: General-Purpose Computing on GPUs

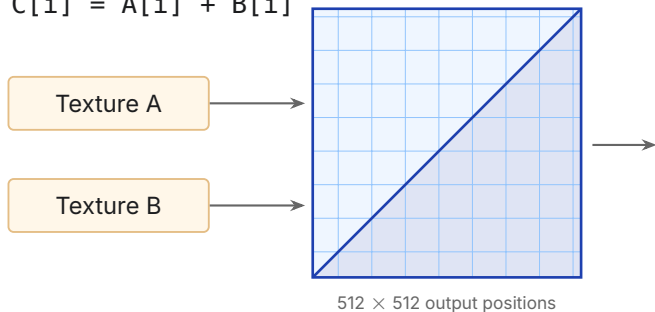
Do those numbers have to be colors?

이 구조를 보던 연구자가 이런 생각을 했다고 해봅시다. 저 숫자가 꼭 색깔이어야 할까? GPU는 입력값을 사람이 물의 색이라고 부르는지, 온도라고 부르는지 신경 쓰지 않습니다. 숫자를 읽고 계산한 다음 결과를 저장할 뿐입니다.

위쪽은 그래픽의 언어입니다. 각 위치의 입력을 셰이더에 넣어 출력색을 만듭니다. 아래쪽에서는 이름만 바꿨습니다. 각 데이터 포인트의 입력을 함수에 넣어 결과값을 만듭니다. 기존 기계를 새로 발명한 것이 아니라, 원래 하던 일에서 그림이라는 의미를 벗겨낸 것입니다. 이런 식으로 많은 데이터에 같은 계산을 병렬로 적용할 수 있다면 그래픽이 아니어도 GPU를 활용할 수 있습니다. 이것이 초기 GPGPU의 핵심 관점입니다.

To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Interpret the output as an array.

A draw call to the GPU; an array operation to the researcher.

To add two arrays, first draw two triangles

발견과 실제 사용 사이에는 재미있는 장애물이 있었습니다. 우리가 하고 싶은 것은 A와 B의 같은 원소를 더해서 C에 쓰는 것뿐인데, GPU에 일을 시키는 통로가 그래픽 API였습니다. 그래서 계산을 그림 그리기로 번역해야 했습니다.

512 곱하기 512개의 원소를 처리하려면 먼저 출력 이미지 크기를 그렇게 설정합니다. 입력 배열은 원래 표면의 무늬를 저장하던 텍스처에 넣습니다. 그다음 출력 영역을 정확히 덮는 삼각형 두 개를 그립니다. 그러면 각 위치에 프래그먼트 셰이더가 실행되고, 셰이더는 색을 만드는 대신 A와 B의 값을 읽어 더합니다. 출력된 이미지를 숫자 배열로 해석하면 계산이 끝납니다. GPU에게는 사각형을 그리는 작업이었지만, 연구자에게는 배열 전체에 함수를 적용하는 작업이었던 것입니다.

“All this just to do A + B”



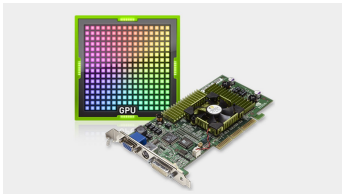
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

“All this just to do A + B”

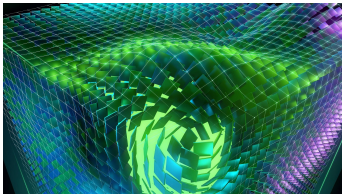
NVIDIA의 2006년 자료도 이 상황을 고작 A 더하기 B를 하려고 이 모든 것을 한다고 표현했습니다. 불편함은 코드가 길어진다는 데서 끝나지 않았습니다.

지금 본 초기 프래그먼트 경로에서는 결과가 그 조각에 대응하는 그래픽 출력 위치로 나갔습니다. 일반 배열의 원하는 원소에 결과를 쓰거나, 다른 실행의 중간값을 주고받는 계산을 표현하는 데 제약이 있었습니다. 입구부터 내부 규칙까지 그림을 그리는 장치에 맞춰져 있었던 것입니다. 그렇다면 이 우회 경로를 정식 계산 인터페이스로 바꿀 수 있지 않을까요?

CUDA arrived with a more general GPU

**1999**

GeForce 256

**2006**

CUDA architecture

launch()

CUDA Toolkit 1.0

2007

CUDA Toolkit 1.0

A compute programming model arrived with a GPU that could execute it.

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

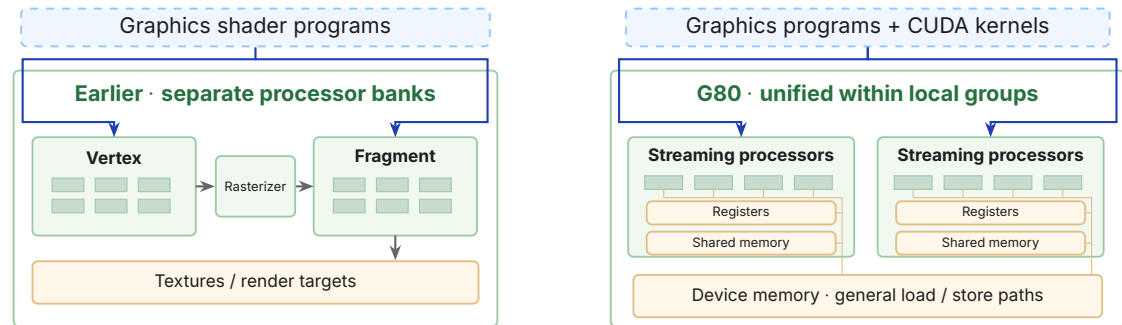
CUDA arrived with a more general GPU

이 우회를 없애려면 일을 요청하는 방법과 그 요청을 처리하는 장치가 함께 바뀌어야 했습니다. 왼쪽의 1999년 GeForce 256은 그래픽 GPU의 이정표입니다. 가운데 2006년에는 G80, 즉 GeForce 8800과 CUDA 아키텍처가 등장했고, 2007년에는 CUDA Toolkit 1.0이 나왔습니다.

G80은 정점용과 픽셀용으로 나뉘던 연산 자원을 통합했습니다. 그래픽에서도 단계별 작업량이 달라질 때 자원을 더 유연하게 쓸 수 있었고, CUDA 계산에도 이 프로세서들을 활용했습니다. CUDA는 이 하드웨어에 계산 프로그램을 직접 제출하는 길을 제공했습니다. GPU에서 실행할 프로그램을 커널, 그 실행 요청을 launch라고 합니다.

단순히 새 API로 삼각형 그리기를 숨긴 것만은 아닙니다. 일반적인 메모리 읽기와 쓰기, 가까운 데이터의 재사용과 실행 간 협력을 지원하는 장치도 중요했습니다. 소프트웨어가 요청하는 일과 하드웨어가 제공하는 기능을 나란히 보겠습니다.

CUDA used unified processors and new memory paths



Selected resources · graphics rasterization remains · processor counts are illustrative

CUDA needed executable instructions, direct data access, and local cooperation.

NVIDIA · GeForce 8800 Technical Brief, pp. 12–14, 21–25 · CUDA Guide 1.0, pp. 2–5

CUDA used unified processors and new memory paths

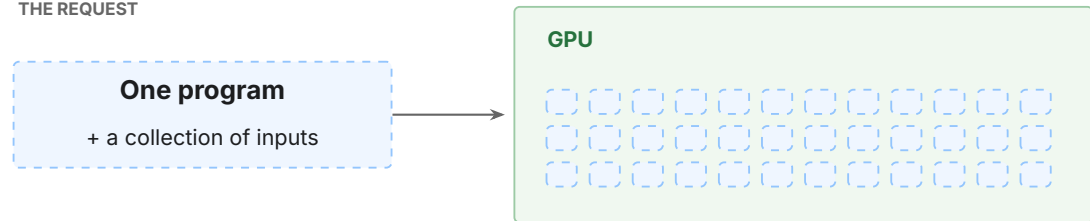
왼쪽은 정점과 프래그먼트 프로그램을 서로 다른 프로세서 뱅크가 처리하던 모습입니다. 가운데 rasterizer가 둘을 연결합니다. 오른쪽 G80은 같은 종류의 프로세서를 그래픽의 여러 단계와 CUDA 계산에 사용할 수 있게 했습니다. 화면에 따라 정점과 픽셀 작업량이 달라지는 그래픽에도 유용한 변화였습니다.

오른쪽은 전 GPU가 하나의 자원을 공유하는 덩어리가 아닙니다. 프로세서와 레지스터, shared memory를 묶은 국소 그룹이 여러 개 있고, 장치 메모리의 일반적인 읽기와 쓰기 경로로 연결됩니다. 선은 연결 관계를 보여주며, 모든 메모리 접근이 shared memory를 거친다는 뜻은 아닙니다. CUDA 커널은 일반 배열에 결과를 쓰고 가까운 데이터를 재사용하며 실행끼리 진행을 맞출 수 있습니다.

이런 실행과 메모리 기능이 있어야 컴파일한 CUDA 명령을 처리할 수 있습니다. 따라서 CUDA 이전 GPU에 Toolkit을 설치하는 것만으로는 부족합니다. 예전에 CUDA를 지원하던 GPU가 최신 Toolkit 지원에서 빠지는 것은 별도의 버전 문제입니다. 이제 이 기계에 하는 요청을 한마디로 줄여보겠습니다.

“Run this program many times.”

THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

“Run this program many times.”

사용자가 이제 하고 싶은 말은 이 프로그램을 많이 실행해달라는 것입니다. 화면을 설정하고 삼각형을 그리라는 말 대신, 실행할 프로그램과 데이터를 직접 건넵니다. 그래픽이라는 외피를 벗겨도 많은 데이터에 같은 계산을 적용한다는 성격은 그대로 남아 있습니다.

여기서 연대기를 마치고 설계 관점으로 바꿔보겠습니다. 이어지는 설명은 실제 발명 순서를 재현하는 것이 아니라, 이런 요청을 처리할 계산기를 만든다면 어떤 구조가 필요할지 생각해보는 과정입니다.

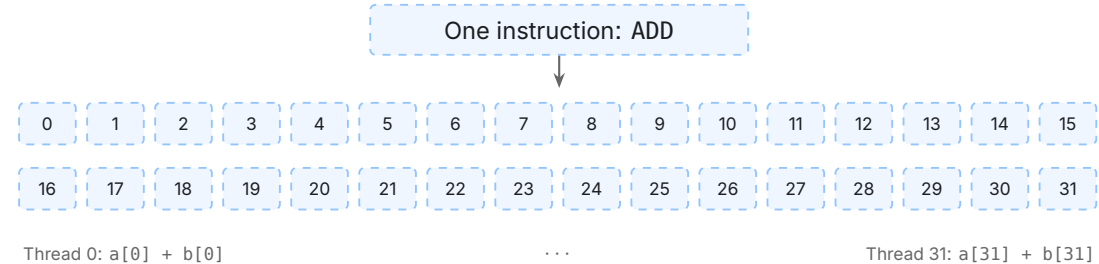
3. The execution engine

- Warps and SIMT
- SMs and GPU organization

The execution engine

이제 NVIDIA CUDA GPU의 실행 엔진을 구성해보겠습니다. Warp와 SM의 역할을 이해한 뒤, AI 작업의 변화가 연산과 메모리, 연결에 무엇을 요구하는지 보겠습니다.

Hardware groups 32 logical threads into a warp



SIMT Single Instruction, Multiple Threads

A warp groups thread execution; execution units do the work.

Hardware groups 32 logical threads into a warp

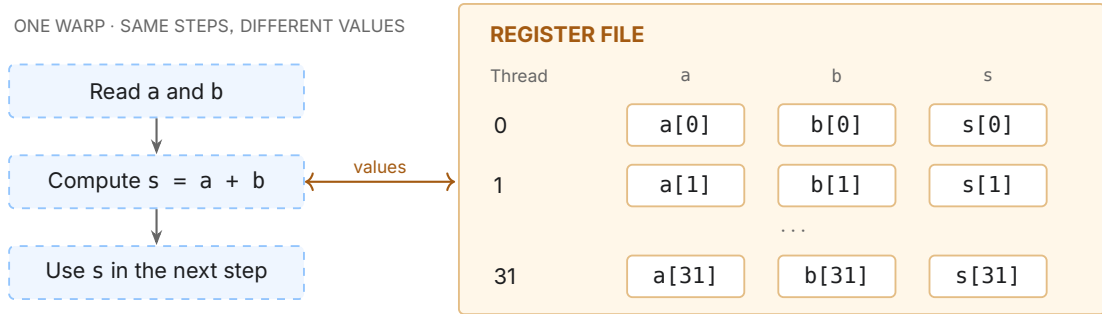
여러 데이터에 대해 덧셈을 해야 한다고 해봅시다. 각 계산의 입력은 다르지만, 지금 해야 할 명령은 모두 덧셈입니다. 하나의 명령으로 여러 데이터의 계산을 함께 진행하면 명령을 전달하고 제어하는 비용을 나눌 수 있습니다.

프로그램의 개별 논리적 실행을 thread라고 합니다. Warp는 하드웨어가 실행을 묶어 관리하는 32개 논리적 thread의 묶음입니다. 여기서 보는 NVIDIA CUDA GPU는 프로그램에서 정하는 더 큰 작업 묶음인 block 안에서, 연속된 번호의 thread를 32개씩 묶습니다. block을 어떻게 구성하는지는 뒤의 코드 예제에서 보겠습니다.

파란 칸은 각 thread의 실행을 나타내며, 특정 연산기와 일대일로 대응하지는 않습니다. 프로그래머는 thread마다 할 일을 쓰고, 하드웨어는 그 thread들의 명령을 묶어 처리합니다. 이것이 SIMT, Single Instruction, Multiple Threads입니다. 조건에 따라 일부 thread만 명령에 참여할 수도 있습니다. 그러면 이 실행들을 진행하려면 하드웨어가 무엇을 기억하고 어떤 일을 해야 할까요?

Each thread needs values that survive an instruction

ONE WARP · SAME STEPS, DIFFERENT VALUES



A register file holds working values; each thread has its own registers.

NVIDIA CUDA Programming Guide · GPU Hardware Model · conceptual register layout

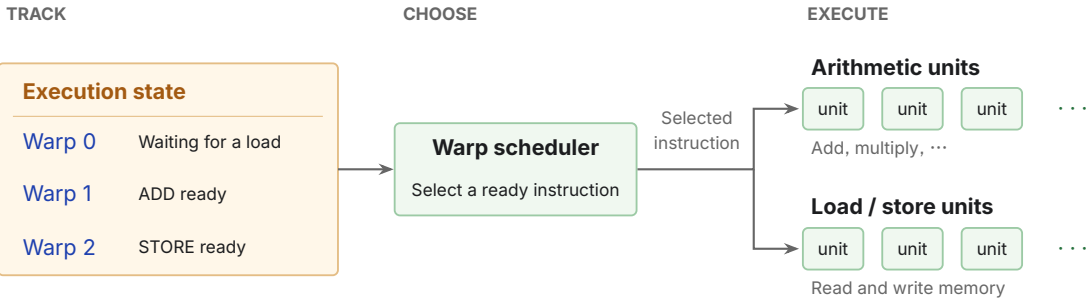
Each thread needs values that survive an instruction

이제 이 thread들의 실행에 필요한 하드웨어를 생각해봅시다. 먼저 덧셈에 쓸 값을 읽었다고 하면, 덧셈 명령이 그 값을 사용할 때까지 어디엔가 보관해야 합니다. 덧셈이 끝나도 다음 계산에서 결과를 다시 쓸 수 있으니, 명령 사이에 값을 남겨둘 자리가 필요합니다.

이런 현재 작업값을 담는 저장 공간이 레지스터입니다. 오른쪽처럼 많은 레지스터를 모아 놓은 하드웨어 저장 구조를 레지스터 파일이라고 부릅니다. 디스크에 저장하는 파일이라는 뜻은 아닙니다. 그림의 가로줄마다 서로 다른 thread의 입력 a, b와 결과 s가 있습니다. 저장 장치는 함께 구성해도, 프로그램에서 각 thread가 사용하는 레지스터 값은 자기 것입니다.

이제 값을 보관할 곳은 생겼습니다. 그런데 여러 warp를 실행하고 있다면, 어떤 실행이 어디까지 왔고 다음에 누구의 명령을 처리해야 하는지도 알아야 하지 않을까요?

Track progress, choose an instruction, execute it



Required roles; unit counts and connections are illustrative.

Track each warp’s state. Issue selected instructions to execution units.

NVIDIA CUDA Programming Guide · Hardware implementation · conceptual roles

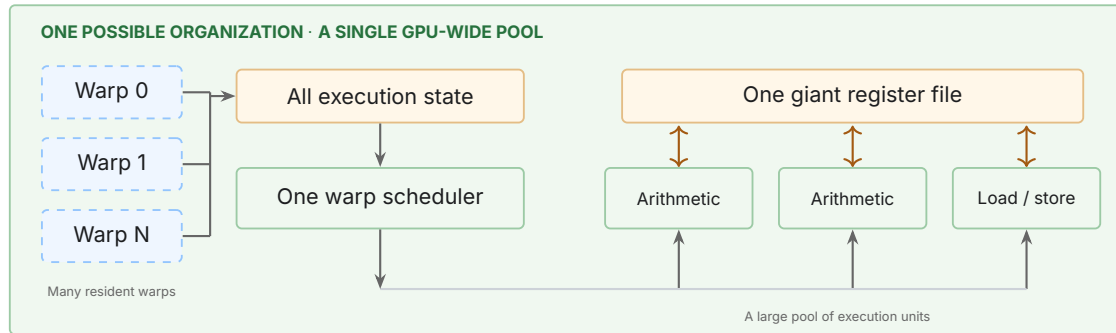
Track progress, choose an instruction, execute it

왼쪽 표는 하드웨어가 warp별로 추적하는 실행 상태의 예입니다. warp 0은 메모리에서 읽는 값을 기다리고, warp 1은 덧셈을 할 준비가 됐고, warp 2는 결과를 쓸 준비가 됐습니다. 레지스터에 담은 작업값과 함께 이런 진행 상황과 준비 여부도 기억해야 합니다.

가운데 warp 스케줄러는 이 상태를 참조해, 필요한 값과 실행 자원이 준비된 명령을 골라 발행합니다. 발행은 화살표에 적힌 선택한 명령을 실행 장치로 보내는 것입니다. 입력값을 기다리는 warp가 있으면 다른 warp의 준비된 명령을 고를 수 있습니다.

오른쪽 위의 산술 연산 유닛들은 덧셈과 곱셈을 하고, 아래의 load/store 유닛들은 메모리 읽기와 쓰기를 처리합니다. 초록 칸과 생략 기호는 여러 유닛을 나타내며, 개수는 예시입니다. 여기까지는 값 보관, 상태 추적, 명령 선택과 실행이라는 역할을 모았습니다. 이제 각 자원을 몇 개 두고 어떻게 연결할지 정해봅시다.

Why not build one giant execution engine?



One place for values. One place to choose work. Many units to execute it.

Why not build one giant execution engine?

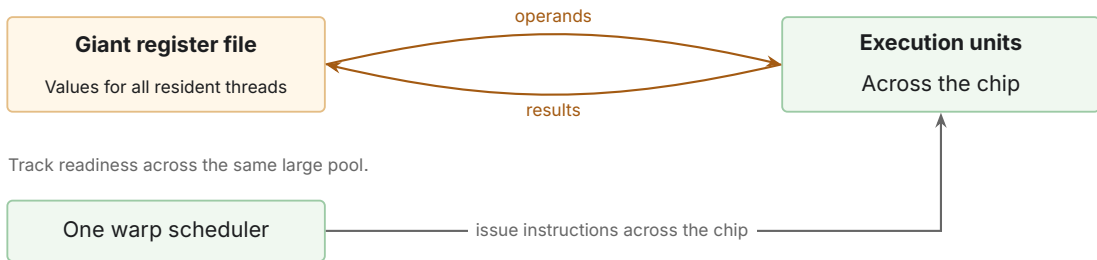
이제 필요한 역할을 실제 부품의 개수와 연결 방식으로 옮겨보겠습니다. 한 가지 안으로 GPU 전체를 하나의 큰 실행 엔진으로 만든다고 상상해봅시다. 많은 warp의 실행 상태를 가운데에서 모두 추적하고, 각 thread의 값은 오른쪽의 거대한 레지스터 파일 하나에 넣습니다. 스케줄러는 하나를 두어 그중 준비된 명령을 골라 아래의 많은 실행 유닛으로 보냅니다.

이렇게 실행 상태와 자원을 배정받아 현재 하드웨어에 머무는 warp를 resident warp라고 합니다. 제출한 모든 작업이 한꺼번에 올라온다는 뜻은 아닙니다. 지금은 올라온 warp들을 한곳에서 관리하는 설계를 생각하는 것입니다. 레지스터 파일 하나를 같이 사용해도 각 thread의 레지스터 값은 여전히 따로 배정됩니다.

얼핏 보면 충분해 보입니다. 일을 더 많이 하려면 레지스터 파일을 키우고, 더 많은 warp와 연산기를 넣으면 되니까요. 그런데 스케줄러가 멀리 떨어진 연산기를 골랐다면, 명령만 보내면 될까요?

The values and control signals must travel too

THE SAME CENTRALIZED DESIGN · NOW CONSIDER THE CONNECTIONS



More units also need more register bandwidth and a larger control network.

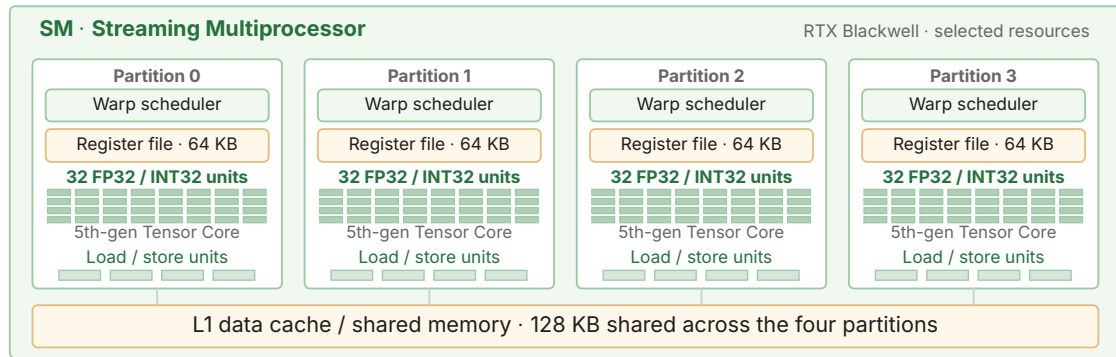
The values and control signals must travel too

스케줄러가 칩 반대편의 빈 연산기를 골랐다고 해봅시다. 명령만 도착한다고 계산할 수는 없습니다. 레지스터 파일에 있는 입력값도 그곳까지 가야 합니다. 계산이 끝나면 결과를 보관할 자리로 돌려보내 다음 명령이 사용할 수 있게 해야 합니다. 위쪽의 주황색 화살표가 그 왕복입니다.

연산기를 더 붙여 여러 명령을 처리하려면, 레지스터 파일도 그만큼 많은 값을 읽고 쓸 수 있어야 하고 그 값을 운반할 연결망도 필요합니다. 아래쪽 스케줄러도 넓은 범위의 준비 상태를 확인하고 여러 실행 유닛에 명령을 전달해야 합니다. 단순히 저장 용량과 연산기 개수만 늘려서는 끝나지 않는 것입니다.

이런 중앙 설계가 불가능하다는 뜻은 아닙니다. 커질수록 함께 해결해야 할 접근 대역폭과 연결 비용이 있다는 뜻입니다. 그렇다면 서로 자주 값을 주고받는 레지스터와 연산기, 이 실행들을 관리하는 장치를 가까이 묶고, 그 묶음을 여러 개 만들면 어떨까요?

An SM keeps execution and its working state close



Each scheduler manages many warps using its partition's local resources.

NVIDIA RTX Blackwell whitepaper · Figure 5, pp. 10–12 · simplified and rearranged

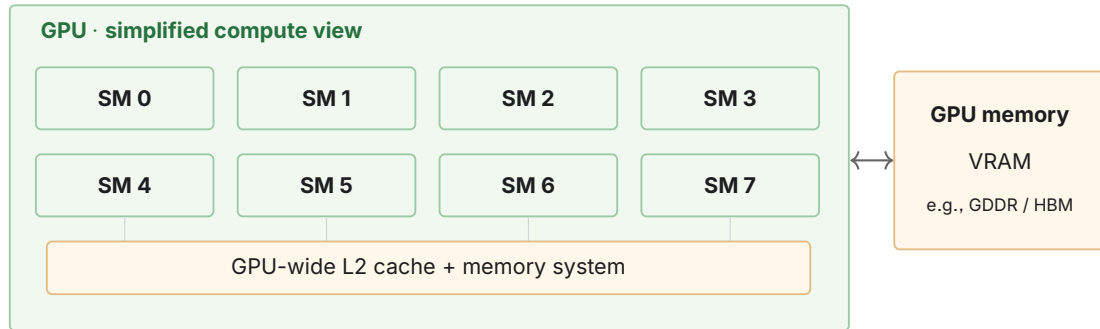
An SM keeps execution and its working state close

앞에서 모은 자원들을 가까이 묶은 실행 엔진이 SM입니다. 내부는 공식 도식이 공개된 RTX Blackwell 기준으로 보겠습니다. 바깥 테두리가 SM 하나이고, 스케줄러와 레지스터 파일, 실행 유닛을 묶은 구획 네 개를 네 열로 펼쳤습니다.

각 스케줄러는 담당 warp들의 준비된 명령을 골라 실행 자원에 공급합니다. 스케줄러 네 개가 warp 네 개를 뜻하지는 않습니다. 아래의 64 KB 레지스터 파일은 thread들의 작업값을 보관합니다. 진한 초록 칸 32개는 FP32 또는 INT32 연산을 하는 유닛으로, 한 사이클에 둘 중 한 종류를 처리합니다. 구획마다 행렬 곱셈과 누적에 특화된 Tensor Core 하나와 load/store 유닛 네 개도 있습니다.

맨 아래 128 KB L1 데이터 캐시와 shared memory는 네 구획이 함께 씁니다. 명령 캐시, dispatch, 특수 함수와 그래픽스 전용 자원은 생략했습니다. 잠시 뒤에 볼 Rubin의 내부 수치로 받아들이면 안 됩니다. 핵심은 SM 안에서조차 자주 주고받는 자원을 가까이 묶는다는 점입니다. 이제 이런 SM을 여러 개 배치해 GPU 전체로 확장해보겠습니다.

Scale the GPU by repeating that execution engine



Share control within a warp; keep execution resources local to an SM.

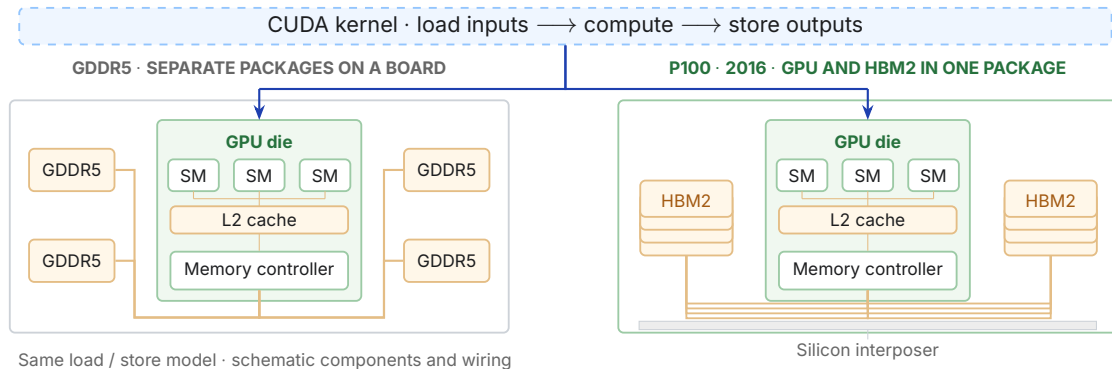
NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

Scale the GPU by repeating that execution engine

이제 자기 몫의 실행을 관리할 수 있는 SM을 여러 개 배치해 GPU를 키워봅시다. 각 SM은 가까운 레지스터와 실행 자원을 사용합니다. 그 바깥에는 여러 SM이 함께 사용하는 L2 캐시와 메모리 시스템이 연결되고, 오른쪽의 큰 GPU 메모리인 VRAM에는 텐서 같은 입력과 출력을 담습니다.

warp에서는 같은 명령의 제어를 함께 쓰고, SM마다 실행 상태와 연산 자원을 가까이 묶습니다. 그런 SM을 반복해서 배치하면 각각 자기 몫의 일을 진행할 수 있습니다. 실제 GPU의 전체 부품을 그린 것은 아니지만, 범용 계산을 담당하는 중심 구조를 이렇게 볼 수 있습니다. 그런데 이 기계에 맡기는 계산이 달라지면, 연산과 메모리에 요구하는 것도 달라집니다. 최신 GPU를 보기 전에 그 변화를 짚어보겠습니다.

More memory traffic changed the GPU package



HBM widens the physical data path; kernels still need efficient access and reuse.

NVIDIA · Inside Pascal (2016) · Dramatic Improvements in Memory · schematic packaging comparison

More memory traffic changed the GPU package

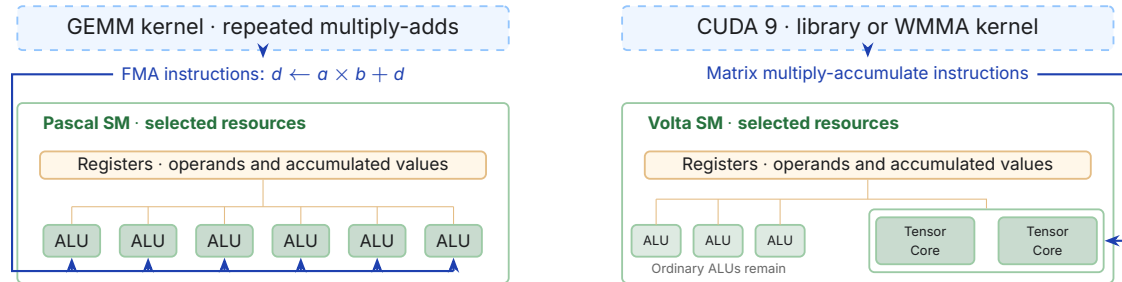
같은 CUDA 커널도 데이터가 도착하지 않으면 기다립니다. 왼쪽은 GPU와 GDDR5 메모리가 보드 위의 별도 패키지로 놓인 구성입니다. GPU 안의 SM이 값을 요청하면 메모리 컨트롤러와 보드의 연결을 통해 데이터를 주고받습니다.

오른쪽 P100은 여러 층으로 쌓은 HBM2를 GPU 다이와 같은 패키지에 놓습니다. 아래의 실리콘 interposer가 둘을 넓게 연결합니다. 여러 갈래로 그린 선은 더 많은 데이터를 공급하는 물리적인 연결을 표현하며 실제 비트 폭이나 층수를 재현한 것은 아닙니다. 큰 데이터를 다루는 과학 계산과 딥러닝에 필요한 대역폭을 높인 변화이고, LLM보다 먼저 사용됐습니다.

맨 위 프로그램은 두 경우 모두 입력을 읽고 계산하고 결과를 씁니다. HBM 전용 CUDA 명령으로 바뀌 쓰는 것은 아닙니다. 하드웨어는 데이터 공급 경로를 넓히고, 커널은 효율적인 접근과 재사용으로 그 대역폭을 활용합니다. 이제 데이터를 받은 뒤 계산하는 회로와 명령 경로도 어떻게 달라졌는지 보겠습니다.

Matrix kernels gained a new physical execution path

MATRIX MULTIPLY + ACCUMULATE · $D = A \times B + C$



Example: FP16 matrix inputs, FP32 accumulation · illustrative unit counts · instruction throughput is not shown

Tensor-aware kernels and instructions use added matrix hardware.

NVIDIA · Programming Tensor Cores in CUDA 9 (2017) · Pascal FMA and Volta Tensor Core paths

Matrix kernels gained a new physical execution path

같은 행렬 곱셈은 왼쪽 GPU도 수행합니다. 커널이 곱셈과 덧셈을 반복하고 일반 ALU들이 FMA, 즉 곱한 뒤 누적하는 명령을 처리합니다. 오른쪽 Volta에는 행렬의 작은 타일을 곱하고 누적하는 Tensor Core와 데이터 경로가 추가됐습니다. 일반 ALU도 남아 있습니다.

새 회로를 추가했다고 기존 커널이 자동으로 이를 사용하는 것은 아닙니다. CUDA 9의 cuBLAS·cuDNN 라이브러리 구현이나 WMMA 행렬 연산 인터페이스를 쓰는 커널이 지원되는 명령 경로를 사용합니다. 이 예시는 FP16 입력과 FP32 누적입니다. 지원하는 자료형과 행렬 조건에 맞는 구현이 필요하고, 단순 반복문이 항상 Tensor Core 명령으로 바뀌는 것은 아닙니다.

파란 코드와 명령의 경로가 달라지고, 초록색 실행 회로도 달라졌습니다. 주황색 레지스터의 입력과 결과가 새 연산 경로로 오갑니다. 이것이 CUDA와 GPU가 함께 발전한 구체적인 모습입니다. 원소별 A 더하기 B는 여전히 일반 산술 경로로 이해할 수 있습니다. 그러면 언어 모델을 실행할 때는 어떤 자원이 더 필요할까요?

Generating text adds a growing state to reuse



Longer context + more requests → more stored state

Generating tokens → repeated access to weights and state

A model spread across GPUs → communication

The bottleneck depends on the model, batch size, and execution phase.

NVIDIA · Mastering LLM Techniques: Inference Optimization · prefill, decode, and KV cache

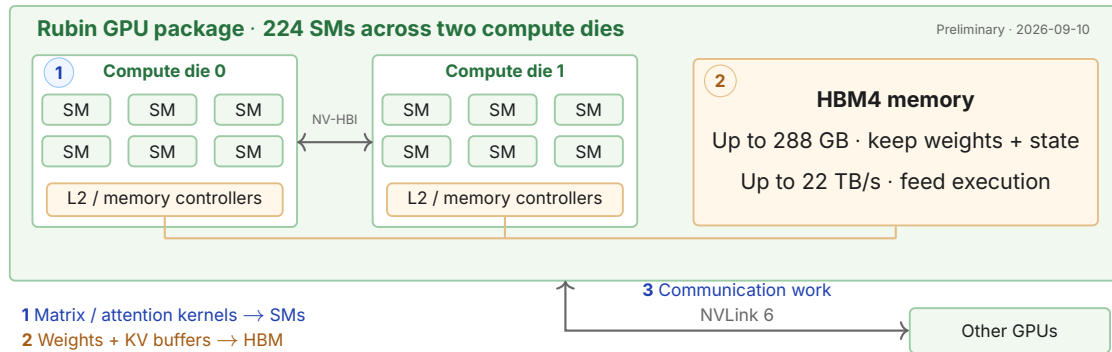
Generating text adds a growing state to reuse

언어 모델은 입력을 읽은 뒤, 앞에서 만든 결과를 바탕으로 다음 토큰을 계속 생성합니다. 입력을 한꺼번에 처리하는 단계와 토큰을 이어 만드는 단계는 기계를 사용하는 방식이 다릅니다.

모델 가중치 외에도 이전 토큰에서 계산한 attention용 값을 보관해 다시 쓰는데, 이것이 KV cache입니다. L1처럼 하드웨어에 붙은 캐시 이름이 아니라 프로그램이 저장하는 데이터입니다. 재계산을 줄이는 대신 메모리 용량과 읽기 대역폭을 사용합니다. 문맥이 길고 동시 요청이 많아지면 이 상태도 커집니다. 특히 작은 배치로 토큰을 생성할 때는 계산량에 비해 읽어야 할 데이터가 많아질 수 있습니다.

모델을 여러 GPU에 나누면 중간 결과를 교환하는 시간도 생깁니다. 따라서 AI는 언제나 메모리가 병목이라고 단정할 수 없습니다. 모델, 배치 크기, 실행 단계에 따라 연산과 메모리, 통신의 비중이 달라집니다. 이제 Rubin을 보면 각 부품이 어떤 요구를 해결하려는지 연결할 수 있습니다.

Rubin scales compute, memory, and communication



The workload shapes the arithmetic, storage, and connections together.

NVIDIA · Inside Rubin GPU Architecture · July 2026 · SM counts, wiring, and software mappings are schematic

Rubin scales compute, memory, and communication

Rubin에서 프로그램과 하드웨어를 연결해보겠습니다. 1번 행렬·attention 커널은 SM에서 실행합니다. 두 compute die에 걸쳐 224개 SM이 일을 나눕니다. Tensor Core뿐 아니라 attention의 참고 비중을 정규화하는 softmax 처리도 강화했습니다. 그림의 SM 상자 수는 예시입니다.

2번 가중치와 KV 버퍼는 HBM에 보관합니다. 최대 288 GB는 상태를 얼마나 담을 수 있는지, 최대 초당 22 TB는 데이터를 얼마나 빨리 공급하는지입니다. 3번 GPU 간 통신 작업은 NVLink 같은 연결을 사용합니다. 두 다이를 한 GPU로 묶는 NV-HBI와 GPU 사이의 NVLink 6는 연결 범위가 다릅니다.

수치는 2026년 9월 10일 기준 공개 예비 사양이고, CUDA 13.4의 Rubin 지원은 sm_107 프리뷰입니다. 앞서 본 RTX Blackwell의 SM 내부 수치를 그대로 적용하면 안 됩니다. GPU가 처리하는 일과 데이터가 바뀌면서 하드웨어와 커널, 라이브러리가 함께 발전합니다. 이제 이 기능들이 우리가 쓰는 코드와 연결되는 생태계를 보겠습니다.

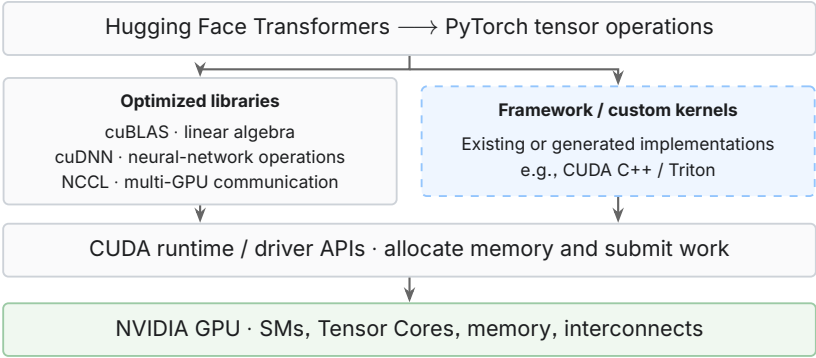
4. Following A + B

- From tensors to threads
- Scheduling and memory
- Results and completion

Following A + B

CUDA 생태계가 프레임워크와 하드웨어를 연결하는 지도를 보겠습니다. 이어 A와 B를 더해 C에 답는 연산으로, 사용자와 GPU가 각각 무엇을 말는지 따라가겠습니다.

CUDA connects frameworks to the hardware



Libraries, kernels, compilers, and tools make hardware capabilities usable.

NVIDIA CUDA platform and libraries · PyTorch / Hugging Face docs · example execution paths

CUDA connects frameworks to the hardware

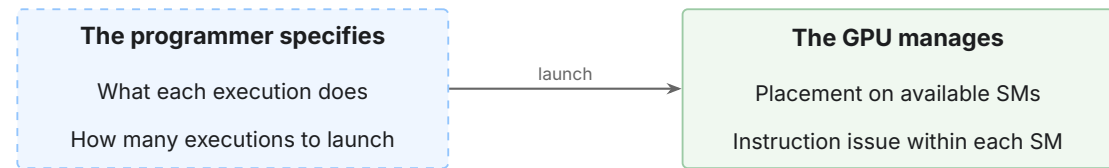
Hugging Face Transformers로 모델을 실행해도 PyTorch를 쓰는 경로에서는 텐서 연산 요청으로 내려 갑니다. 구현은 연산과 실행 조건에 따라 달라집니다.

왼쪽의 cuBLAS는 선형대수, cuDNN은 신경망 연산, NCCL은 GPU 사이의 통신을 제공합니다. 이들을 모두 순서대로 거치는 것은 아닙니다. 오른쪽처럼 프레임워크가 가진 커널이나 직접 작성하고 생성한 커널을 쓸 수도 있습니다. 런타임과 드라이버 API는 메모리 준비와 실행 제출을 연결합니다.

옆의 nvcc는 실행할 코드를 만드는 컴파일러입니다. 매 연산마다 반드시 컴파일한다는 뜻은 아니니 다. Nsight Systems는 프로그램 전체의 시간 흐름을, Nsight Compute는 개별 커널을 들여다봅니다. 하드웨어의 새 기능을 사용자 코드의 성능으로 연결하려면 이런 구현과 도구도 함께 발전해야 합니다.

이제 큰 지도에서 원소별 덧셈 하나를 골라 따라가겠습니다. A 더하기 B가 이 라이브러리들을 모두 호출하는 것은 아닙니다.

Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

A work description is not a physical placement plan.

Describe the work; the GPU places it

이제 사용자의 입장에서 보면, CUDA 프로그래머의 몫은 할 일의 구성을 표현하는 것입니다. 각 실행이 무엇을 할지, 그 실행들을 어떻게 묶어서 얼마나 실행할지 정합니다. 어느 SM이 그 일을 맡을지는 GPU가 사용 가능한 자원에 맞춰 결정하고, 각 SM이 명령을 실행합니다.

PyTorch를 쓰면 커널을 고르고 실행 규모를 정하는 과정도 프레임워크가 맡습니다. 우리는 텐서에 대한 연산을 요청합니다. 다음 코드의 한 줄에서 시작해, 그 요청이 방금 본 SM과 warp에 어떻게 도착하는지 따라가보겠습니다.

Start with one line of PyTorch

```
import torch
A = torch.randn(1024, 1024, device="cuda")
B = torch.randn(1024, 1024, device="cuda")
C = A + B
```

A

+

B

=

C

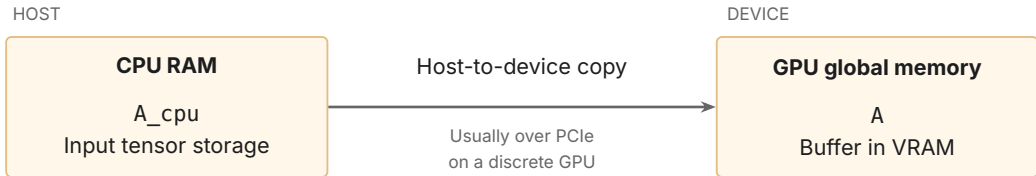
Each output needs only its own two inputs. No neighbor must finish first.

Start with one line of PyTorch

GPU에 있는 1024 곱하기 1024 행렬 두 개를 더하는 코드입니다. C는 A와 B의 같은 위치에 있는 원소끼리 더한 결과입니다. 행렬 전체를 하나의 숫자로 합치는 연산이 아니라, 같은 크기의 행렬을 만드는 원소별 덧셈입니다.

여기서 첫 번째 덧셈이 끝나야 두 번째 덧셈을 할 수 있을까요? 그렇지 않습니다. 각 결과에 필요한 것은 자기 위치의 입력 두 개뿐이고, 옆 원소의 결과를 기다릴 필요가 없습니다. 따라서 백만 개가 넘는 덧셈을 여러 실행에 나눠 맡길 수 있습니다. 처음에 보았던 많은 데이터에 같은 계산을 반복하는 구조입니다. 이 계산을 실행하기 전에, 먼저 입력이 어디에 준비되는지 확인하겠습니다.

If the input starts in CPU memory, copy it to the GPU



```
A_cpu = torch.randn(1024, 1024, device="cpu")  
A = A_cpu.to("cuda")
```

With `device="cuda"`, our original example already creates GPU tensors.

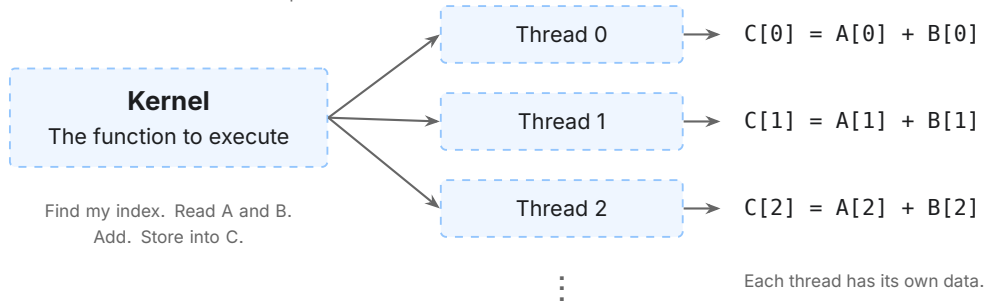
If the input starts in CPU memory, copy it to the GPU

원래 코드는 device에 cuda를 지정해 처음부터 GPU 텐서를 만듭니다. 여기서는 입력이 CPU에 있는 경우만 잠깐 보겠습니다. 파일에서 읽거나 CPU에서 만든 데이터는 처음에 왼쪽 RAM에 있습니다. 아래 첫 줄은 그런 출발점을 보여주고, 다음 줄의 `to cuda`는 그 값을 GPU 쪽 저장 공간으로 옮깁니다. 일반적인 별도 그래픽카드에서는 이 복사가 PCIe 연결을 지나고, 데이터는 VRAM에 있는 GPU의 global memory 버퍼에 놓입니다. B도 CPU에서 출발한다면 같은 방식으로 준비합니다.

이 입력 복사와, 커널이 실행되며 GPU 안에서 값을 읽는 동작은 별개입니다. 후자의 경로는 뒤에서 읽기 명령을 따라가며 보겠습니다. 어느 쪽에서 출발했든 이제 덧셈에 사용할 A와 B는 GPU에 준비되어 있습니다.

One program, many logical executions

Educational kernel · one element per thread · matrix elements numbered as a flat buffer



A kernel is code. A thread is one logical execution of that code.

CUDA Programming Guide · Kernel and thread model

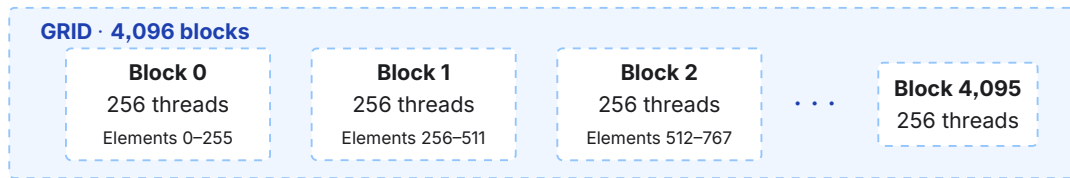
One program, many logical executions

이제 GPU에 준비된 입력으로 돌아와, 같은 덧셈을 thread 하나가 원소 하나를 맡는 교육용 커널로 표현하겠습니다. 실제 PyTorch는 한 thread에 여러 원소를 맡기는 등의 최적화를 사용할 수 있으므로, 여기서 고르는 실행 설정과는 다를 수 있습니다.

먼저 행렬의 원소를 행 순서대로 일렬로 번호 매겨, 1차원 버퍼처럼 보겠습니다. 이 그림에서 A의 0번은 첫 번째 행 전체가 아니라 맨 첫 원소이고, A의 1번은 그다음 원소입니다. GPU에서 실행할 함수가 커널이고, 그 함수를 실행하는 논리적인 인스턴스 하나가 thread입니다. 커널에는 자기 원소 번호를 알아내고, A와 B를 읽고, 더해서 C의 같은 위치에 저장하라는 내용을 씁니다. 왼쪽에는 프로그램이 하나만 있고, 오른쪽의 thread들은 각자 다른 원소 번호로 그 프로그램을 실행합니다. 같은 코드를 여러 번 실행하되, 각 실행이 서로 다른 데이터를 맡는 것입니다.

How many executions do we launch?

$1024 \times 1024 = \mathbf{1,048,576}$ elements



```
// CUDA C++: A/B/C buffers are ready; device_a/b/c are their GPU addresses.
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

4,096 blocks $\times$ 256 threads = 1,048,576 logical threads.

CUDA Programming Guide · Thread blocks and grids

How many executions do we launch?

우리 행렬은 1024 곱하기 1024이므로 출력 원소가 104만 8576개입니다. 한 thread에 한 원소를 맡기면 논리적인 실행도 그만큼 필요합니다. CUDA는 thread를 block으로 묶고, 이번 커널 실행의 block 전체를 grid라고 부릅니다. 여기서는 block 하나에 thread 256개를 넣겠습니다. 이 크기는 우리가 고른 값입니다. 첫 block은 0번부터 255번까지, 다음 block은 256번부터 511번까지를 처리하므로 block은 4096개 필요합니다.

아래는 이 실행 요청을 표현한 CUDA C++ 코드입니다. 결과를 쓸 같은 크기의 GPU 버퍼 C도 미리 준비했다고 하겠습니다. device_a, device_b, device_c는 A, B, C 버퍼의 주소입니다. 꺾쇠 안의 두 숫자는 add 커널을 block 4096개로 실행하되 각 block에 thread 256개를 넣으라는 뜻입니다. 이렇게 커널과 실행 규모를 지정해서 제출하는 것이 kernel launch입니다. 여기까지 정한 것은 전체 작업의 구성이고, 실제 SM 배치는 GPU에 맡깁니다.

Every thread can find its own element

```
__global__ void add(
    const float* a,
    const float* b,
    float* c,
    int element_count
) {
    int index = blockIdx.x * blockDim.x
               + threadIdx.x;
    if (index < element_count) {
        c[index] = a[index] + b[index];
    }
}
```

Same code, different built-in IDs, different data.

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

778

C[778] = A[778] + B[778]

The bounds check protects the final block when the size is not divisible by 256.

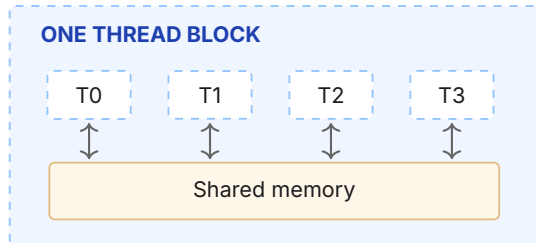
__global__: GPU kernel .x: 1D launch IDs

Every thread can find its own element

방금 호출한 add의 함수 내용입니다. 첫 줄의 global 표시는 GPU에서 실행할 커널이라는 뜻입니다. 소문자 a, b, c는 앞의 device_a, device_b, device_c 주소를 받습니다. 같은 코드에서 각자 맡은 원소를 구분할 때는 CUDA가 제공하는 번호를 사용합니다. blockIdx는 자기 block의 번호, blockDim은 한 block의 크기, threadIdx는 block 안에서 자기 번호입니다. 점 x는 이번에 사용하는 1차원 실행의 번호나 크기를 가리킵니다.

block 번호에 block 크기를 곱하면 그 block의 시작 원소 번호가 되고, 여기에 자기 thread 번호를 더합니다. 오른쪽의 block 3, thread 10은 3 곱하기 256에 10을 더한 778번입니다. 앞서 일렬로 붙인 원소 번호이므로, A와 B의 778번 원소를 읽어 C의 778번에 저장합니다. if문은 원소 수가 block 크기로 나누어떨어지지 않을 때 마지막 block의 불필요한 접근을 막습니다. 이렇게 각 thread가 자기 번호로 담당 데이터를 찾습니다.

A block is also a cooperation boundary



Share data locally

Threads in a block can exchange data.

Synchronize within the block

`__syncthreads()` provides a block-wide barrier.

For example, threads can combine intermediate results through shared memory.

This addition needs no shared intermediate values or block barrier.

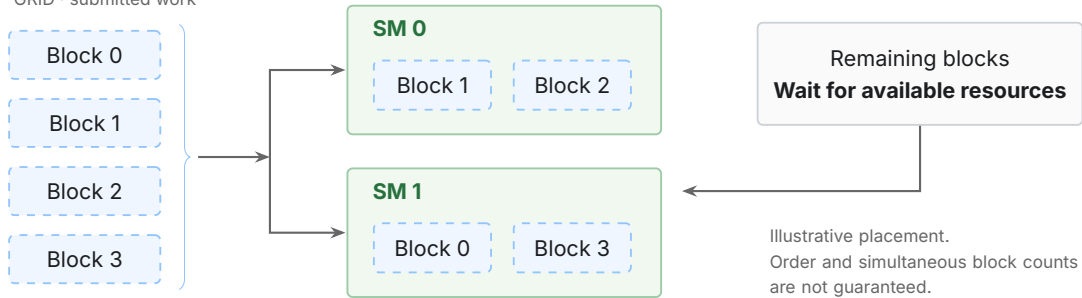
A block is also a cooperation boundary

block에는 실행을 묶는 것 외에도 역할이 있습니다. 같은 block의 thread들은 shared memory라는 가까운 저장 공간으로 데이터를 주고받을 수 있습니다. 또 `syncthreads`라는 동기화 지점에서 서로 기다렸다가 진행할 수 있습니다. 여러 thread가 각자 계산한 중간값을 모아 합치는 경우를 생각해볼 것입니다. 값을 shared memory에 써두고, 모두 쓸 때까지 기다린 뒤 다른 thread의 값도 읽는 식으로 협력할 수 있습니다.

이번 원소별 덧셈에는 자기 입력 두 개만 있으면 됩니다. 서로의 중간값을 읽거나 기다릴 일이 없으므로 이 협력 기능을 사용하지 않습니다. 이제 준비한 block들을 실제 실행 엔진에 배치해보겠습니다.

Many blocks share a finite set of SMs

GRID · submitted work



A block stays on one SM. Multiple blocks let work spread across SMs.

CUDA Programming Guide · Assigning blocks to SMs

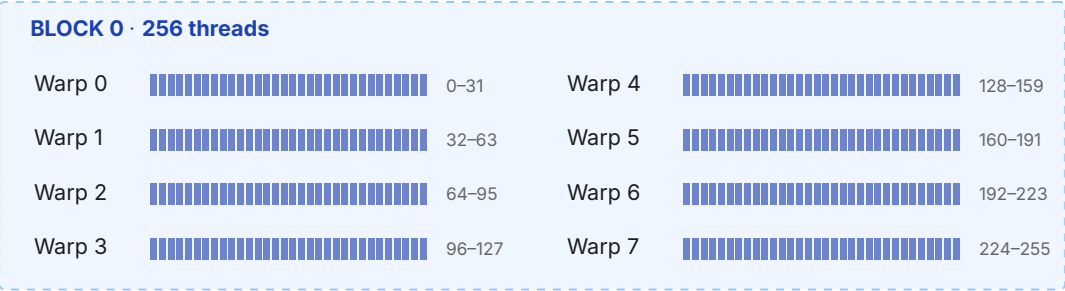
Many blocks share a finite set of SMs

GPU는 제출한 grid의 block들을 SM에 배치합니다. 왼쪽의 작업 묶음이 가운데 실행 엔진에 올라가는 모습입니다. 그림은 가능한 배치의 예이고, 실제 순서와 동시에 올라가는 block 수는 GPU와 커널에 따라 달라집니다.

Block 하나는 같은 SM에서 실행됩니다. 따라서 block 안의 thread 수를 늘려도 그 block이 여러 SM에 나뉘지는 않습니다. 여러 SM에 일을 주려면 여러 block을 제출해야 합니다. 이번에 준비한 block 4096개를 GPU가 여러 SM에 나눠 맡기는 것입니다.

SM 하나에는 레지스터, shared memory, 실행 상태를 보관할 공간이 허용하는 만큼 여러 block이 함께 올라갈 수 있습니다. 자원이 차면 나머지 block은 기다리고, 실행 중인 block이 끝나 자리가 나면 다음 block이 배치됩니다. 이렇게 더 적은 수의 SM이 자원을 되쓰며 전체 작업을 처리합니다.

A 256-thread block contains eight warps



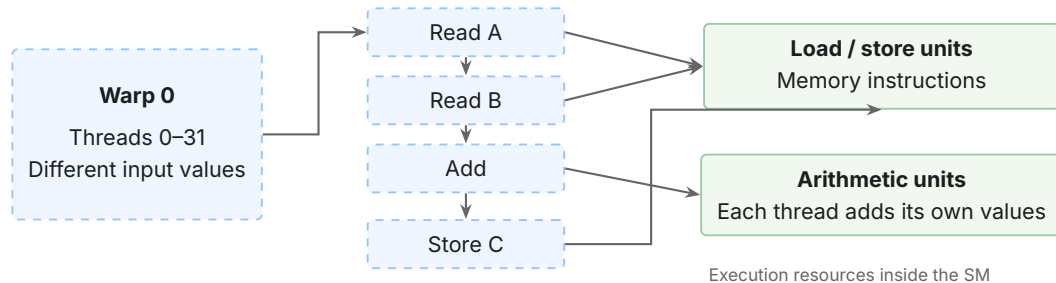
Within a block, consecutive groups of 32 threads form warps.

A 256-thread block contains eight warps

SM에 올라간 block 0을 확대해보겠습니다. 작은 파란 막대 하나가 thread 하나입니다. NVIDIA GPU는 같은 block 안에서 연속된 번호의 thread를 32개씩 묶어 warp로 관리합니다. 첫 warp는 thread 0번 부터 31번, 다음 warp는 32번부터 63번을 말합니다. 이런 식으로 마지막 warp는 224번부터 255번을 말합니다.

우리는 256개 thread를 한 block으로 제출했고, SM은 이를 여덟 warp로 나누어 관리합니다. 이제 첫 번째 warp가 덧셈을 수행하는 과정을 가까이서 보겠습니다.

A warp needs different units for different instructions



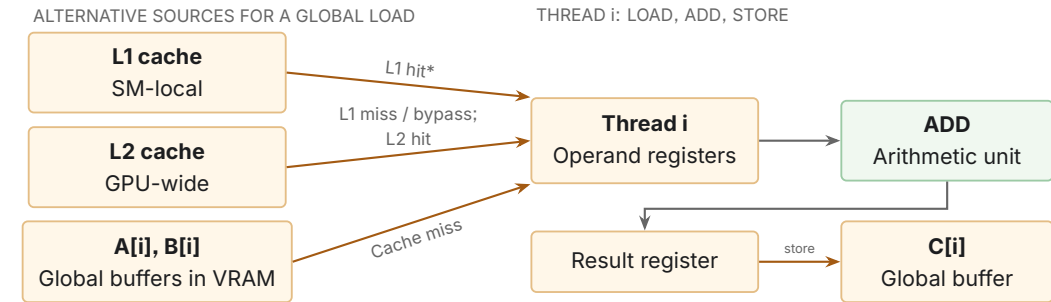
The instruction is shared; the data belongs to each thread.

A warp needs different units for different instructions

warp 0의 thread들이 해야 할 일은 같습니다. 자기 A 값을 읽고, 자기 B 값을 읽고, 둘을 더해 자기 C 위치에 저장합니다. 덧셈 명령이 발행되면 0번 thread는 A와 B의 0번 원소를, 31번 thread는 각 배열의 31번 원소를 더합니다. 명령은 함께 처리하고 데이터는 각자 맡는 SIMT가 이 계산에서도 나타납니다.

오른쪽을 보면 읽기와 쓰기는 load/store 유닛을 이용하고, 덧셈은 산술 연산 유닛을 사용합니다. 그림은 주소 계산 등의 세부 명령을 생략하고 네 종류의 작업에 집중했습니다. 각 명령은 그 종류에 맞는 SM의 자원을 사용합니다. 먼저 이 명령에 필요한 값이 어디에 보관되는지 보고, 이어 여러 warp의 명령이 번갈아 진행되는 모습을 보겠습니다.

For this addition, operands are loaded into thread registers



*L1 use depends on architecture and policy. Arrows show possible data sources, not physical routes.

The add kernel uses per-thread registers. Shared-memory staging is unnecessary.

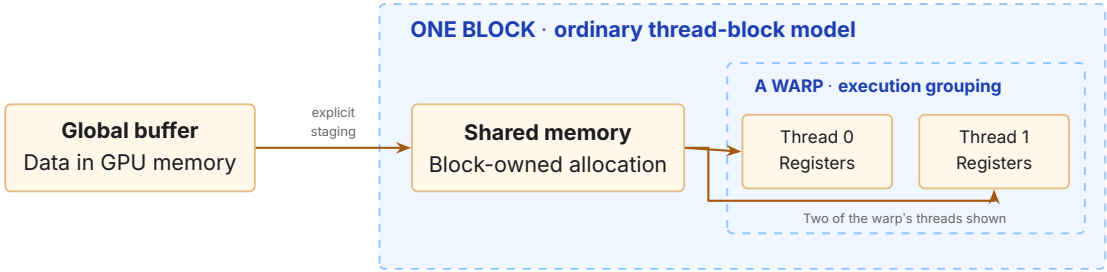
NVIDIA PTX ISA · Cache operators · logical data flow, not a physical floorplan

For this addition, operands are loaded into thread registers

GPU에 준비해 둔 A와 B의 값은 실제 ADD 명령까지 어떻게 도착할까요? thread는 자기 원소를 읽는 global load 명령으로 값을 가져와 작업용 레지스터에 둡니다. 왼쪽의 세 화살표는 이 읽기에 값을 줄 수 있는 공급원을 보여줍니다. L1을 사용하는 읽기에서 값이 L1에 있으면 거기서 얻습니다. L1에 없거나 L1을 사용하지 않는 읽기라면 L2가 처리할 수 있고, 캐시에도 값이 없으면 VRAM에서 가져와야 합니다. L1 사용 여부는 아키텍처와 명령의 정책에 따라 달라집니다. L2는 여러 SM이 공유하고, L1은 각 SM 안에 있는 캐시입니다.

어느 곳이 읽기를 처리하든 계산에 사용할 값은 오른쪽 thread의 레지스터에 놓입니다. 산술 연산 유닛이 두 값을 더하면 결과도 레지스터에 생기고, global store가 C의 해당 위치에 씁니다. 쓰기 경로는 별도의 캐시 정책을 따를 수 있습니다. 레지스터 값은 각 thread에 속하고, 물리적으로는 SM의 레지스터 파일에 보관됩니다. 이번 덧셈에는 이 값들을 shared memory로 따로 옮길 필요가 없습니다.

Shared memory is explicit storage for block cooperation



Useful when threads reuse or exchange data. This addition does not take this optional path.

Shared memory is optional staging that the kernel manages for cooperation and reuse.

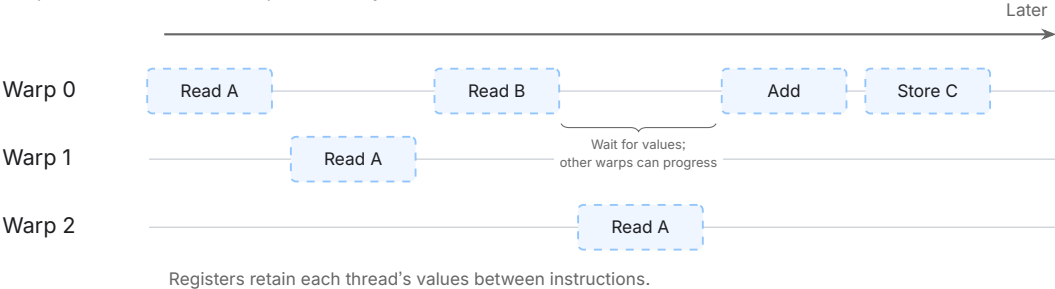
Shared memory is explicit storage for block cooperation

앞서 block 협력에 쓴다고 했던 shared memory를 이번에는 데이터 경로에서 보겠습니다. 캐시는 하드웨어가 관리하지만, shared memory에는 커널이 재사용하거나 주고받을 데이터를 명시적으로 준비합니다. 프로그램이 데이터를 쓰고 필요한 동기화를 거친 뒤, 각 thread가 자기 레지스터로 값을 읽어 계산합니다. 그림의 화살표도 두 thread의 레지스터에 닿습니다. 점선 warp 경계는 실행 묶음을 표시하며, 별도의 메모리 계층을 추가하지 않습니다.

L1과 shared memory가 같은 물리 저장 자원을 나누어 쓰는 GPU도 있지만, 프로그램에서 쓰는 방식은 다릅니다. shared memory는 협력과 재사용이 필요한 커널이 선택하는 저장 공간입니다. 이번 덧셈으로 돌아오면 앞 장의 레지스터 경로를 사용합니다. 이제 값이 오기를 기다리는 동안 SM이 다른 warp의 명령을 어떻게 진행시키는지 보겠습니다.

The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



A warp does not have to finish its whole program before another runs.

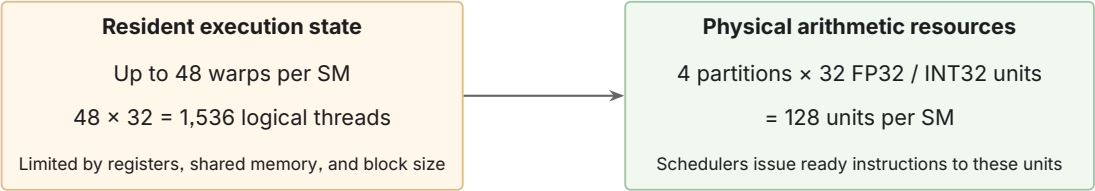
The scheduler issues the next ready instruction

스케줄러는 준비된 warp의 다음 명령을 골라 발행합니다. warp 하나를 끝낸 뒤에야 다른 warp를 시작하는 방식은 아닙니다. 그림은 가능한 발행 순서의 예입니다. warp 0이 A를 읽는 명령을 내보낸 뒤 warp 1도 A를 읽고, 이어 warp 0의 B 읽기와 warp 2의 A 읽기가 진행됩니다. warp 0이 값을 기다리는 동안에도 다른 warp의 준비된 명령을 발행할 수 있습니다. 값이 준비되면 warp 0은 더하고 C에 저장합니다.

그 사이 각 thread의 값은 레지스터에 보관됩니다. 실행 상태를 유지하면서 준비된 일을 고르는 것, 이것이 SM에 레지스터와 스케줄러를 연산기 가까이 둔 이유입니다.

Many resident warps share the same execution units

RTX BLACKWELL · COMPUTE CAPABILITY 12.0



Resident threads retain their state; they do not each own an arithmetic unit.
Warp width, instruction throughput, and result latency are separate quantities.

Resident warp capacity is not the arithmetic-unit count divided by 32.

NVIDIA Blackwell Tuning Guide · Occupancy (CC 12.0); RTX Blackwell whitepaper · SM architecture

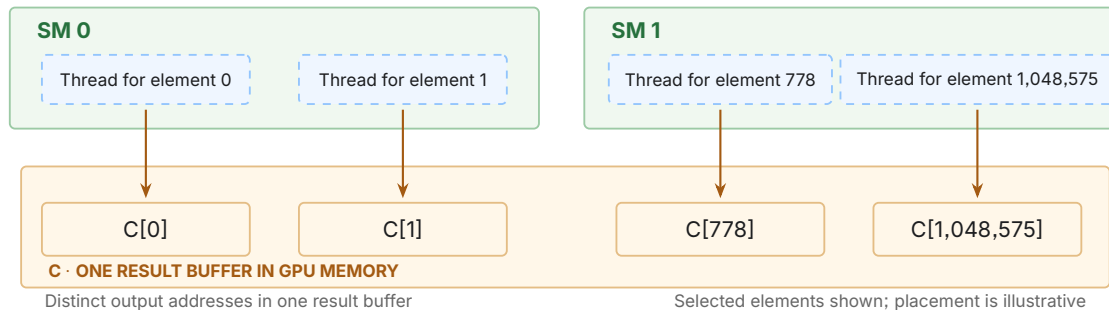
Many resident warps share the same execution units

방금 본 스케줄링을 실제 자원 수와 연결해보겠습니다. 앞의 SM 도식과 같은 RTX Blackwell, compute capability 12.0 기준입니다. SM 하나에는 최대 48개 warp, 즉 1536개 논리 thread의 실행 상태가 동시에 머물 수 있습니다. 실제로 몇 개가 머무는지는 레지스터와 shared memory 사용량, block 크기 같은 조건에 따라 줄어듭니다.

반면 FP32와 INT32 연산을 처리하는 물리 유닛은 네 구획에 32개씩, SM 전체에 128개입니다. 1536개 thread가 각자 전용 연산기를 갖는 것이 아닙니다. 값과 실행 상태를 보관해 두고, 스케줄러가 준비된 명령들을 골라 같은 연산 자원을 사용하게 합니다. 따라서 resident warp 수를 연산기 수 나누기 32로 계산하면 안 됩니다.

Warp의 32는 함께 명령을 실행하도록 관리하는 논리 thread의 수입니다. 특정 명령을 단위 시간에 얼마나 처리하는지, 결과가 다음 명령에 쓰일 때까지 얼마나 기다려야 하는지는 별개의 질문입니다. 이 숫자만으로 모든 명령이 한 사이클에 끝난다고 해석할 수도 없습니다.

Threads write directly into C



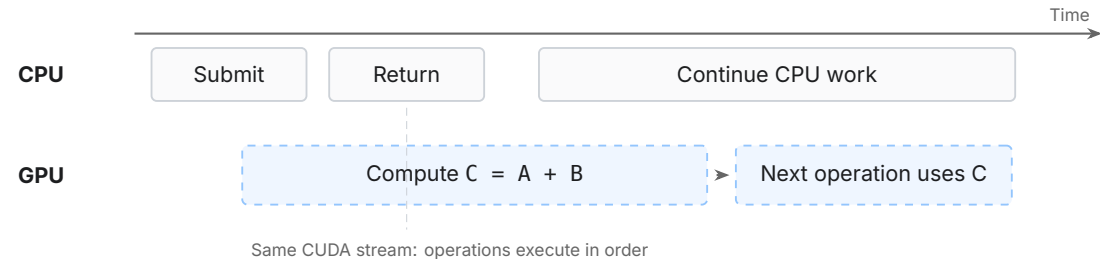
Each thread stores its own result. The CPU does not reassemble the matrix.

Threads write directly into C

이렇게 진행한 덧셈의 결과가 어디에 저장되는지 보겠습니다. 각 thread는 자기 원소를 계산한 다음, launch 전에 준비해 둔 GPU 메모리의 C에 직접 씁니다. 그림의 주황색 공통 경계가 하나의 결과 버퍼이고, 그 안에서 서로 다른 출력 위치를 말합니다. 몇 개의 원소만 골라 그린 것입니다.

모든 thread의 저장이 끝나면 그 버퍼에 결과 행렬이 완성됩니다. 각 thread가 처음부터 최종 위치에 쓰므로 CPU가 SM별 조각을 받아 다시 조립할 필요도 없습니다. 그러면 Python에서 C를 받았을 때 이 값들은 모두 준비되어 있을까요?

Python can return before the GPU finishes



- GPU operations are normally submitted asynchronously.
- Ordinary CPU copies wait as needed; GPU timing must account for completion.

PyTorch documentation · CUDA semantics · illustrative timeline

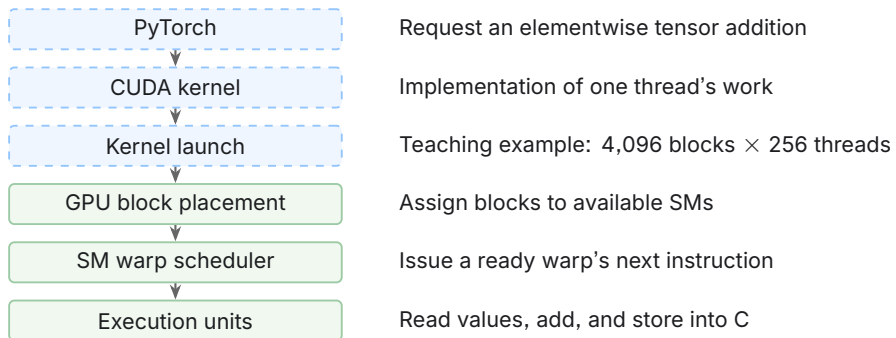
Python can return before the GPU finishes

Python은 결과를 담을 GPU 버퍼를 가리키는 텐서 객체 C를 받을 수 있지만, 그 버퍼의 값은 아직 계산 중일 수 있습니다. PyTorch의 GPU 연산은 기본적으로 비동기적으로 제출되기 때문입니다. 그림에서 CPU는 덧셈을 제출한 뒤 다음 일을 진행하고, GPU는 그 아래에서 계산을 계속합니다. C를 받는 시점과 값이 완성되는 시점을 나눠서 보면 됩니다.

CUDA stream은 GPU 작업을 순서대로 제출하는 통로입니다. 그림처럼 같은 stream에 C를 사용하는 다음 연산을 제출하면 앞선 덧셈 뒤에 실행되므로, 매번 CPU가 기다릴 필요는 없습니다. 서로 다른 stream 사이의 의존성은 별도로 관리합니다.

CPU로 값을 가져오는 일반적인 `C.cpu()` 복사는 PyTorch가 필요한 기다림을 수행합니다. 반면 GPU 계산 시간을 측정할 때는 완료 시점까지 포함하도록 동기화나 CUDA 이벤트를 사용해야 합니다. 호출 전후의 CPU 시간만으로는 GPU가 실제로 계산한 시간을 알 수 없습니다.

Follow the request all the way down



One tensor operation becomes many scheduled instructions on real hardware.

Follow the request all the way down

이제 한 줄의 요청과 실제 실행을 연결해보겠습니다. 이 그림은 위에서 아래로 작업의 표현, 구현, 하드웨어 실행의 관계를 보여줍니다. PyTorch에서는 A와 B를 더해 C에 담은 텐서 연산을 요청합니다. 그 연산을 GPU에서 수행하는 것이 CUDA 커널 구현입니다. 우리가 따라온 교육용 구현은 한 thread가 한 원소를 처리하며, block 4096개와 block당 thread 256개로 실행했습니다.

아래 초록색 부분에서는 GPU가 block을 자원이 있는 SM에 배치하고, SM의 스케줄러가 준비된 warp의 명령을 발행합니다. 입력은 GPU 버퍼에서 각 thread의 레지스터로 읽히고, 산술 유닛에서 더한 결과는 C의 담당 위치에 직접 기록됩니다. 이 저장까지 모두 끝나는 시점은 Python 호출의 반환 시점과 다를 수 있습니다. 한 줄의 요청이 여러 실행과 명령으로 나뉘어, 하나의 결과 버퍼를 완성하는 것입니다.

Keep the work and the machine separate

Kernel	The GPU program to execute
Thread	One logical execution of that program
Block	A group of threads that can cooperate
Grid	All blocks submitted for this kernel launch
Warp	A 32-lane execution group within one block
SM	The hardware engine that executes resident warps

Grid ≠ GPU. Block ≠ SM. Thread ≠ a dedicated core.

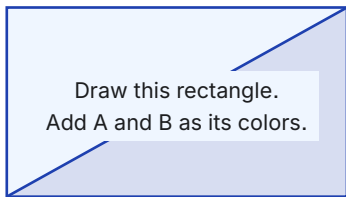
Keep the work and the machine separate

마지막으로 일과 기계의 관계 세 가지만 짚겠습니다. grid는 제출한 작업 전체이고, GPU는 그것을 처리하는 기계입니다. block은 한 SM에 배치되지만, 그 SM의 자원은 여러 block이 함께 쓸 수 있습니다. thread는 개별 실행이고, warp로 묶여 명령마다 필요한 하드웨어 자원을 사용합니다.

이 구별을 처음의 질문에 적용해보면, 데이터가 VRAM에 있다는 사실과 여러 SM이 계산하고 있다는 사실은 별개입니다. nvidia에서 메모리를 많이 사용한다고 해서 연산 자원도 그만큼 바쁘다고 볼 수는 없습니다. 또 프레임워크의 Python 호출이 돌아오는 것과 GPU 계산이 끝나는 것도 다릅니다. 이제 메모리에 둔 데이터, 제출한 작업, 실제 실행을 나누어 볼 수 있습니다.

From two triangles to one line

THEN



NOW

$$C = A + B$$

The constant idea: apply the same computation to many inputs.

From two triangles to one line

처음의 삼각형 두 개로 돌아왔습니다. 예전에는 이 사각형을 그려달라고 말하면서, 각 위치의 색은 A와 B를 더해서 정하라고 해야 했습니다. 지금은 프레임워크에서 A와 B를 더해 C에 담아달라고 요청하면 됩니다.

그 사이에 GPU가 활용하던 성질 자체가 사라진 것은 아닙니다. 많은 데이터에 같은 계산을 적용한다는 일은 계속 같았습니다. 달라진 것은 계산을 시키기 위해 그림을 그리는 척할 필요가 없어졌고, 그 일을 표현하는 도구와 실제로 처리하는 하드웨어가 함께 발전했다는 점입니다. AI가 요구하는 행렬 계산과 데이터 이동은 Tensor Core와 HBM 같은 자원의 발전으로도 이어졌습니다. CUDA의 라이브러리와 컴파일러는 그 기능을 사용자의 코드에 연결합니다. 오늘 본 thread와 block, warp와 SM은 그 한 줄과 실제 계산기를 연결하는 언어입니다. GPU를 다음에 사용하실 때 이 구조를 떠올릴 수 있으면 좋겠습니다. 감사합니다.

마지막으로 이 자료의 설명 범위를 말씀드리겠습니다. 오늘 본 커널과 다이어그램은 기본 개념을 이해하기 위한 교육용 예시입니다. 실제 하드웨어의 세부 구조와 메모리 동작, 실행 설정은 GPU 아키텍처와 소프트웨어 버전에 따라 달라질 수 있습니다. 특히 예제의 block 수와 thread 수를 실제 PyTorch 내부의 고정된 구현으로 받아들이지는 않으시면 좋겠습니다. NVIDIA와 Stanford의 자료는 출처를 표시하고 참고한 것이며, 이 발표에 대한 공식적인 보증이나 지지를 뜻하지 않습니다. 설명에 들어간 해석과 오류의 책임은 발표자에게 있습니다. 질문이나 수정 의견은 아래 연락처로 보내주시면 감사하겠습니다.

Educational scope This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.

Beyond one GPU: choose a question

	AMD + ROCm HIP TPU
Other accelerators	Torch/XLA PJRT StableHLO NPU startups Cerebras
LLMs and the data center	Training Serving Parallelism Reliability, power, cooling NVIDIA networking
Bits, bandwidth, packaging	Number formats Mixed precision HBM FlashAttention TSMC packaging
Linux and deployment	Torvalds and NVIDIA Portable GPU images Containers and the host driver

The recurring question: what must change around the compute units?

Optional reference topics · click a topic or use Appendix index · sources checked September 10, 2026

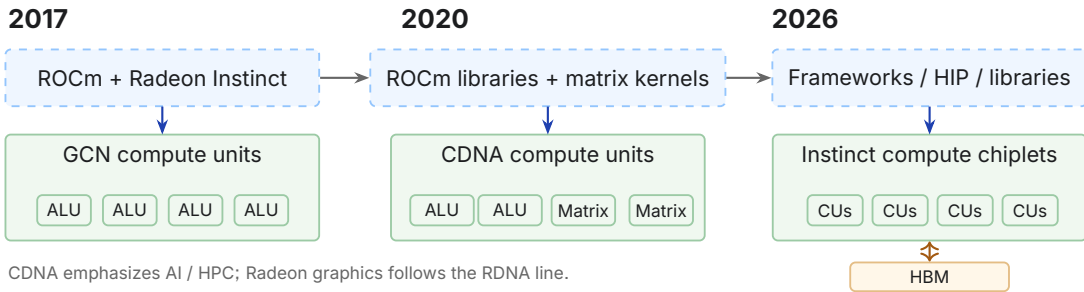
Beyond one GPU: choose a question

여기부터는 질문에 맞춰 골라 볼 수 있는 부록입니다. 다른 가속기는 어떤 선택을 했는지, LLM을 큰 규모로 돌릴 때 무엇이 달라지는지, 숫자와 메모리·패키징은 왜 중요해졌는지, 마지막으로 Linux와 컨테이너가 하드웨어를 실제 사용 환경에 어떻게 연결하는지 다룹니다.

모두 연산기 자체만으로는 설명할 수 없는 문제입니다. 원하는 모델과 서비스가 정해지면 컴파일러, 데이터 이동, 물리적인 조립 방식, 운영체제 경계까지 함께 맞아야 합니다. 화면의 주제를 누르면 해당 장으로 이동하고, 각 장 위의 Appendix index를 누르면 이 목록으로 돌아옵니다.

AMD built a compute platform alongside its GPUs

A FEW MILESTONES · HARDWARE AND SOFTWARE TOGETHER



The alternative is a hardware, compiler, library, and tooling stack.

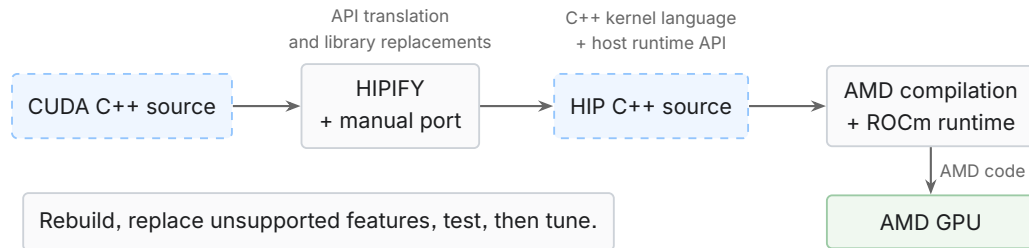
AMD 2017 results · CDNA white paper, pp. 2-7 · CDNA architecture · illustrative units

AMD built a compute platform alongside its GPUs

그동안 AMD도 GPU로 범용 계산을 하고 있었습니다. 2017년 Radeon Instinct에도 ROCm 소프트웨어를 함께 제공했고, 2020년 CDNA에서는 기존 스칼라·벡터 연산에 행렬 실행부를 더했습니다. 그래픽에 집중한 RDNA와 데이터센터 계산에 집중한 CDNA가 갈라진 것이지요.

오른쪽의 오늘날 Instinct도 계산 칩렛과 HBM만으로 완성되지 않습니다. 프레임워크가 라이브러리와 커널을 선택하고 컴파일러와 런타임이 실제 장치에 연결해줘야 합니다. 그래서 AMD의 대응은 GPU 제품 하나가 아니라 ROCm이라는 소프트웨어 플랫폼을 함께 만드는 일이었습니다. 그 안에서 CUDA 경험을 가져오는 통로가 HIP입니다.

HIP moves source code to a different GPU stack



Source portability does not imply CUDA binary or performance compatibility.

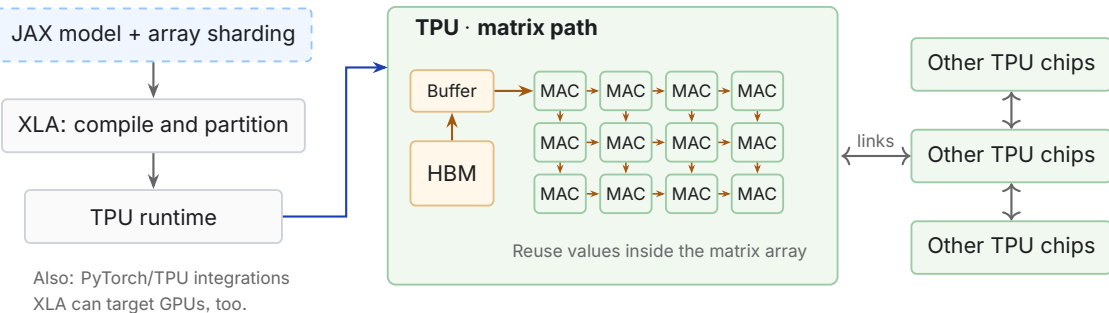
AMD · What is HIP? · C++ runtime API, kernel language, HIPIFY, and porting limits

HIP moves source code to a different GPU stack

HIP는 Heterogeneous-compute Interface for Portability의 약자입니다. ROCm에서 GPU 커널 언어와 메모리·실행을 제어하는 호스트 API를 제공합니다. CUDA C++와 닮아 있어서 HIPIFY로 많은 호출을 변환할 수 있습니다.

하지만 이미 빌드한 CUDA 실행 파일을 AMD GPU가 그대로 읽는다는 뜻은 아닙니다. 그림처럼 소스를 옮겨 AMD용으로 다시 컴파일합니다. 지원하지 않는 API나 라이브러리를 바꾸고, 정확성을 확인한 뒤 하드웨어에 맞게 튜닝해야 합니다. 프레임워크가 이 작업을 대신해주는 경우도 있지만, 사용자 정의 CUDA 커널이 있으면 이식 작업이 다시 드러납니다.

TPUs start from neural-network dataflow



The compiler maps tensor work and communication onto an ML-focused machine.

Google · TPU architecture · JAX stack · TPU, ISCA 2017 · selected datapath, illustrative array and topology

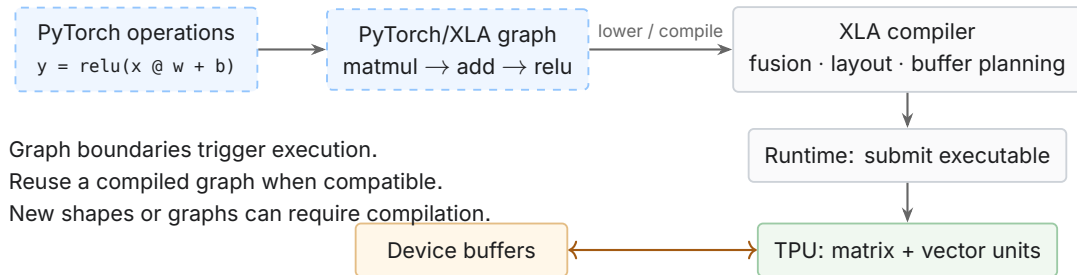
TPUs start from neural-network dataflow

TPU, Tensor Processing Unit은 Google이 머신러닝을 위해 설계한 전용 프로세서입니다. 서비스 추론의 지연시간과 전력 비용을 줄이는 데서 출발해, 학습 처리량과 대규모 연결로 확장했습니다. 행렬 실행부에서는 곱셈·누산 회로 사이로 값을 흘려보내며 재사용합니다. 물론 메모리와 다른 종류의 연산부도 필요합니다.

왼쪽은 JAX의 대표 경로입니다. 배열 계산과 분할 방식을 XLA가 컴파일해 TPU와 여러 칩 사이에 배치합니다. PyTorch 연동도 있고 XLA는 GPU도 지원합니다. 차이는 특정 언어의 독점이 아니라 출발점입니다. 그래픽에서 범용 커널로 확장한 GPU에 비해 TPU는 텐서 계산·컴파일러·분산 시스템을 중심으로 발전했습니다.

PyTorch/XLA first turns tensor work into a graph

CLASSIC LAZY-TENSOR PATH · BEFORE AND AFTER THE RUNTIME CHANGE



The tensor API looks familiar; the compiler chooses how that work fits the TPU.

PyTorch/XLA 2.4 · XLA Tensor Deep Dive · classic path, not every PyTorch/TPU execution mode

PyTorch/XLA first turns tensor work into a graph

PyTorch를 쓰면서도 TPU를 사용할 수 있게 한 연결이 PyTorch/XLA입니다. 여기서는 전통적인 lazy tensor 경로를 보겠습니다. 행렬 곱, 덧셈, 활성화 함수를 호출하면 먼저 계산 그래프에 기록하고, 그래프 경계나 결과가 필요한 시점에 컴파일한 일을 실행합니다. XLA는 여러 연산의 결합과 데이터 배치, 버퍼 사용을 결정해 TPU의 행렬·벡터 실행부에 맞춥니다.

같은 조건의 그래프는 컴파일 결과를 재사용하지만, 모양이나 그래프가 달라지면 새 컴파일 비용이 생길 수 있습니다. 이것은 TPU에서 Python의 모든 호출이 그대로 하나의 하드웨어 명령이 된다는 뜻이 아닙니다. 다음의 PJRT 전환은 이 그래프 계산 자체를 처음 만든 사건이 아니라, 컴파일과 실행을 장치에 요청하는 연결을 바꾼 사건입니다. 또한 2026년에는 TorchTPU라는 더 네이티브한 통합 방향도 발표됐으므로, 이 그림을 모든 시대의 PyTorch·TPU 실행 방식으로 일반화하지 않겠습니다.

PJRT replaced the device-runtime connection



PyTorch/XLA 2.0: default when no XRT configuration · 2.1: stable · JAX also uses PJRT

Less runtime plumbing; a shared API for devices, buffers, compilation, and execution.

PyTorch/XLA · XRT to PJRT, 2023 milestones · OpenXLA · uniform device API · schematic runtime boundaries

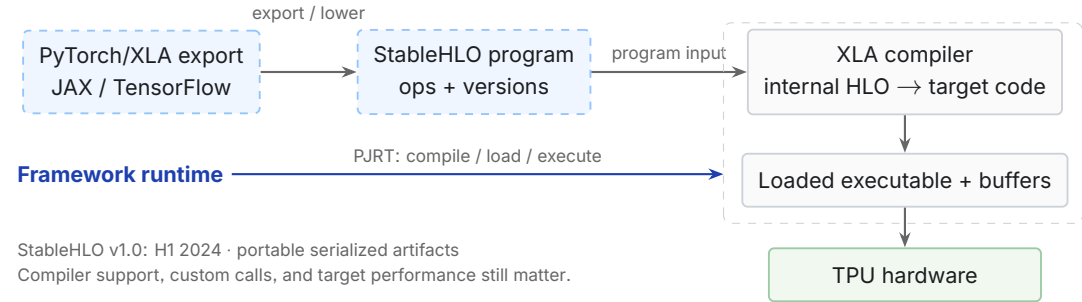
PJRT replaced the device-runtime connection

이전에는 PyTorch/XLA가 TensorFlow 기반 XRT 서비스에 gRPC로 요청을 보내 TPU의 컴파일과 실행을 사용했습니다. 이 별도 서비스 경계는 설정과 요청 처리 비용을 만들었습니다. PJRT로 옮기면서 JAX도 사용하는 장치 API를 통해 TPU용 libtpu 구현에 접근하게 됐습니다. PyTorch/XLA 2.0에서는 기존 XRT 설정이 없을 때 기본 선택이 됐고, 2.1에서 안정화됐습니다.

PJRT는 장치 목록과 메모리 버퍼, 컴파일 요청, 실행과 완료를 다루는 인터페이스입니다. XLA 컴파일러를 없앤 것도, 새 TPU 연산기를 만든 것도 아닙니다. 공급사는 이 인터페이스 뒤에 장치별 구현을 연결할 수 있습니다. gRPC 경로의 부담과 실행 설정은 줄지만, 여러 호스트의 데이터 교환과 동기화가 사라지는 것은 아닙니다. 여기까지는 장치에 어떻게 일을 요청하는가의 변화입니다. 다음에는 그 요청에 답을 계산을 어떤 언어로 표현하는가를 보겠습니다.

StableHLO stabilizes the program, PJRT runs it

BEFORE: HLO / MHLO WERE NOT A VERSIONED PORTABLE CONTRACT



StableHLO defines what to compute; PJRT provides the device API used to compile and run it.

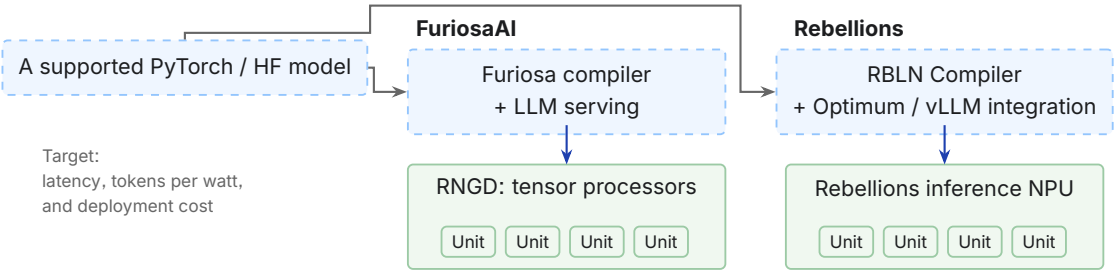
StableHLO · compatibility · PyTorch export · PJRT API · selected TPU path; other compilers can consume StableHLO

StableHLO stabilizes the program, PJRT runs it

XLA의 계산 표현인 HLO와 그 MLIR 표현인 MHLO는 있었지만, 오래 저장해 다른 버전의 도구와 교환할 안정적인 계약은 별도 문제였습니다. StableHLO는 연산의 의미를 명시하고 버전과 직렬화 호환성을 마련한 계산 표현입니다. 2024년 상반기에 1.0이 나왔습니다. PyTorch export와 PyTorch/XLA로 만든 표현을 전달할 수 있고, JAX와 TensorFlow도 생산자가 될 수 있습니다. XLA 외에 이를 받는 컴파일러도 있습니다.

파란 윗길은 무엇을 계산할지 담은 프로그램이고, 아래 화살표는 컴파일·버퍼·실행을 요청하는 PJRT API입니다. 둘은 앞뒤 세대의 대체재가 아니라 서로 다른 경계입니다. XLA 내부에서는 여전히 HLO로 최적화하고 목표 장치용 코드를 만듭니다. StableHLO 파일 자체가 TPU 기계어나 모델 가중치 전체인 것은 아닙니다. 호환성 보장도 정해진 직렬화와 범위에 적용되며, 지원 연산과 custom call, 장치별 성능까지 자동으로 같아지는 것은 아닙니다. 다른 가속기를 붙일 때도 이런 계산 표현과 실행 구현 양쪽이 필요합니다.

NPU startups compete on the complete inference service



Target:
latency, tokens per watt,
and deployment cost

Check the supported operators, shapes, quantization, and serving features for the actual model.

Specialized hardware needs a compiler and serving stack that keep up with models.

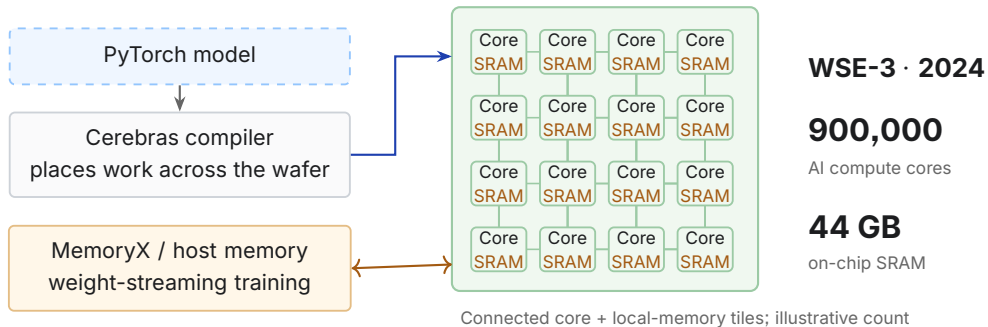
Furiosa SDK 2026.2 · Rebellions Inference Platform · examples of inference-focused NPUs; conceptual units

NPU startups compete on the complete inference service

NPU는 Neural Processing Unit, 신경망 계산용 프로세서라는 넓은 이름입니다. 모두 같은 구조도, 모두 추론 전용도 아닙니다. 여기서는 데이터센터 추론에 집중하는 두 회사를 봅니다. Furiosa는 RNGD와 컴파일러·서빙 엔진을, Rebellions는 NPU와 RBLN 컴파일러·프레임워크 연동을 함께 만듭니다.

목표는 같은 서비스를 요구 지연시간 안에서 더 적은 전력과 비용으로 제공하는 것입니다. 하지만 새로운 모델을 실제로 실행하려면 연산, 텐서 모양, 숫자 형식, KV 관리까지 소프트웨어가 지원해야 합니다. 그래서 칩의 최대 연산량만 비교하면 핵심을 놓칩니다. 실제 모델의 품질과 지연시간, 처리량을 함께 확인해야 합니다.

Cerebras makes the wafer itself a compute fabric



Move much more compute and memory onto one fabric; the system boundary still matters.

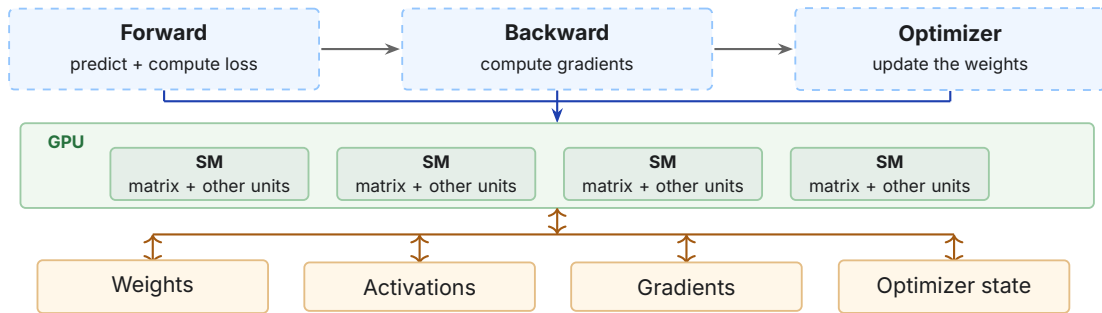
Cerebras · WSE-3 announcement, March 2024 · PyTorch compiler · Weight streaming

Cerebras makes the wafer itself a compute fabric

보통은 웨이퍼를 개별 칩으로 잘라냅니다. Cerebras는 그 경계까지 연결해서, 웨이퍼 규모의 연결망에 계산 코어와 SRAM을 반복 배치합니다. 2024년 WSE-3는 90만 개 코어와 44GB의 칩 내부 SRAM을 가진 예입니다. GPU의 HBM 용량과는 메모리 종류와 위치부터 다릅니다.

컴파일러는 모델의 일을 이 거대한 공간에 배치합니다. 그렇다고 모델과 데이터가 언제나 웨이퍼 안에 전부 들어간다는 뜻은 아닙니다. 큰 모델의 학습에는 외부 MemoryX에서 가중치를 흘려보내는 방식도 씁니다. 계산과 저장을 가까이 놓는 선택을 극단까지 밀어붙였지만, 외부 메모리·입출력·전력·냉각을 포함한 시스템 설계는 여전히 필요합니다.

Training keeps much more than the model weights



HBM holds tensor state. Sharding spreads state across GPUs; recomputation can save activations.

Fitting the weights is only the start of fitting a training job.

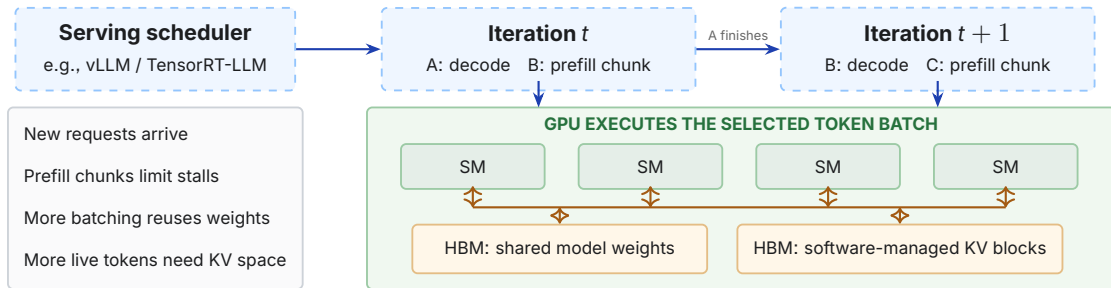
Rajbhandari et al. · ZeRO (2019/2020), §§3–4 · original schematic; physical counts illustrative

Training keeps much more than the model weights

LLM 학습은 예측 한 번으로 끝나지 않습니다. Forward에서 예측과 손실을 계산하고, backward에서 가중치를 얼마나 바꿀지 gradient를 구한 뒤, optimizer가 갱신합니다. 세 단계는 같은 GPU의 실행 자원을 사용하지만 필요한 커널이 다릅니다.

HBM에는 가중치 외에도 중간 activation, gradient, optimizer의 이력 상태가 필요합니다. 정확한 양은 학습 방법과 숫자 형식에 따라 달라집니다. ZeRO 같은 방식은 상태를 여러 GPU에 나눠 담고, activation 재계산은 일부 저장 공간을 계산 시간과 맞바꿉니다. 그래서 모델 파일이 GPU 메모리에 들어간다고 그대로 학습할 수 있는 것은 아닙니다.

LLM serving schedules tokens and memory together



Illustrative schedule. The best mix depends on model, context, hardware, and latency target.

Serving balances time to first token, time between tokens, throughput, and memory.

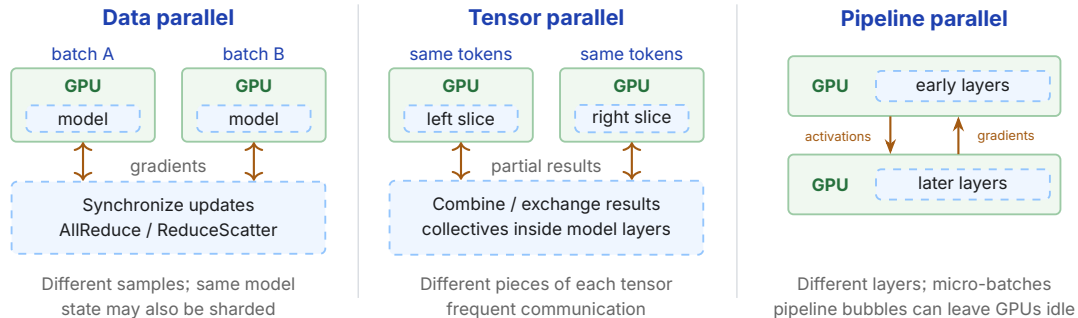
Kwon et al. · PagedAttention / vLLM (2023) NVIDIA · TensorRT-LLM chunked prefill (2024)

LLM serving schedules tokens and memory together

서비스에서는 길이가 다른 요청이 계속 들어옵니다. 그림에서 A가 끝나면 다음 반복에 C를 넣습니다. 긴 B의 입력도 작은 prefill 조각으로 나누면 이미 답변 중인 요청이 오래 기다리는 일을 줄일 수 있습니다.

여러 요청을 묶으면 가중치를 읽어 더 많은 토큰의 계산에 재사용할 수 있습니다. 대신 살아 있는 요청의 KV 상태를 더 많이 보관해야 합니다. vLLM과 TensorRT-LLM 같은 실행기는 토큰 스케줄과 메모리 관리를 함께 다룹니다. KV는 SM의 하드웨어 캐시가 아니라 HBM 등에 저장하는 소프트웨어 상태입니다. 첫 답변까지의 시간, 토큰 사이 간격, 전체 처리량을 동시에 좋게 만드는 단일 batch 크기는 없습니다.

Splitting a model creates different kinds of traffic



Parallelism must fit both GPU memory and the network topology.

Llama Team · The Llama 3 Herd of Models (2024), §3.3.2 · original schematic; these strategies can be combined

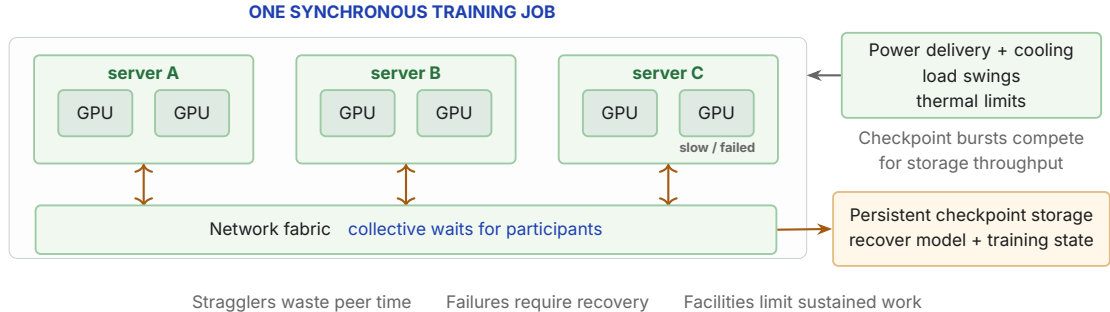
Splitting a model creates different kinds of traffic

GPU를 늘리는 방법도 여러 가지입니다. Data parallel은 서로 다른 샘플을 계산하고 gradient를 합쳐 갱신을 맞춥니다. 상태를 나눠 보관하는 방식도 결합할 수 있습니다.

Tensor parallel은 행렬 같은 tensor 자체를 나눠 계산하므로 모델의 층 안에서도 부분 결과가 자주 오갑니다. Pipeline parallel은 층을 나눠 맡아 forward의 activation과 backward의 gradient를 전달합니다. 작은 묶음을 흘려도 앞뒤 단계를 기다리는 빈 시간이 생길 수 있습니다.

실제 학습은 이런 방법을 조합합니다. 같은 GPU 수라도 자주 통신하는 작업을 멀리 떨어뜨리면 느려지므로, 소프트웨어의 분할 방식과 물리 연결을 함께 결정해야 합니다.

At cluster scale, one slow component can stall the job



Useful training time depends on the whole facility, including recovery.

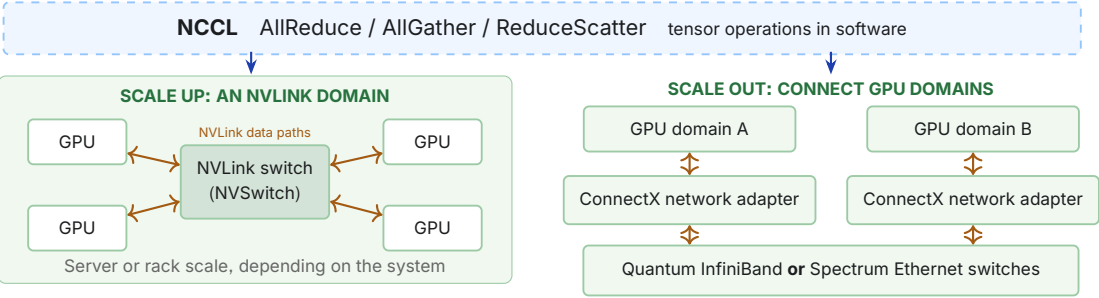
Llama Team · The Llama 3 Herd of Models (2024), §§3.3.1, 3.3.4 · reported production-cluster challenges

At cluster scale, one slow component can stall the job

동기식 학습에서는 한 GPU나 연결이 느려져도 함께 통신하는 GPU들이 기다릴 수 있습니다. 고장이면 저장해둔 학습 상태로 복구해야 하고, 상태를 자주 저장하면 손실은 줄지만 스토리지에 큰 쓰기 부하가 생깁니다.

Meta의 Llama 3 논문에는 이런 문제가 실제 운영 경험으로 나옵니다. GPU들이 계산하다 동시에 통신이나 저장을 기다리면 데이터센터 전력 사용량도 함께 바뀝니다. 온도에 따른 동작 속도 변화 역시 전체 처리량에 영향을 줬습니다. 그래서 대규모 학습은 GPU 개수만의 문제가 아닙니다. 장애 탐지와 복구, 네트워크, 저장 장치, 전력과 냉각이 실제로 학습하는 시간을 결정합니다.

NVIDIA also builds the paths between GPUs



Mellanox joined NVIDIA in 2020. BlueField DPUs offload networking, storage, and infrastructure services.

Fast GPUs need fast communication; NCCL chooses how tensor data moves over hardware.

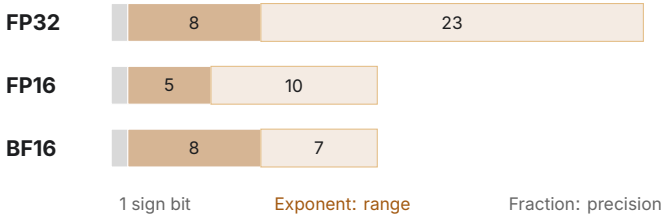
NVIDIA also builds the paths between GPUs

GPU가 데이터를 기다리는 시간을 줄이려면 GPU 사이 연결도 개선해야 합니다. 왼쪽 NVLink와 NVSwitch는 서버나 랙 안에서 긴밀한 GPU 연결 영역을 만듭니다. 오른쪽은 ConnectX 같은 어댑터와 InfiniBand 또는 Ethernet 스위치로 그 영역들을 연결합니다.

맨 위 NCCL은 통신 프로토콜이나 칩이 아니라, tensor를 합치고 모으는 집단 통신 라이브러리입니다. 실제 장비의 연결에 맞춰 데이터를 옮깁니다. NVIDIA는 2020년 Mellanox 인수를 마쳤고, 네트워킹 제품군을 함께 갖추게 됐습니다. BlueField는 네트워크와 스토리지 등 인프라 처리를 맡습니다. 개별 GPU의 계산 성능을 전체 서비스의 처리량으로 연결하는 범위까지 넓어진 것입니다.

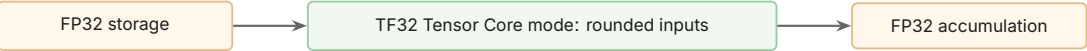
A number’s bits buy range, precision, and cost

STORED FLOATING-POINT VALUES · equal width per bit



Same 16 bits, different choices
BF16 keeps FP32’s exponent range.
FP16 keeps more fraction bits.

More bits still have a role
FP64 for demanding numerical work;
FP32 for sensitive calculations.



Precision and range are different. TF32 is a compute mode for FP32 data.

NVIDIA Transformer Engine · format comparison · NVIDIA · TF32 internals (2021)

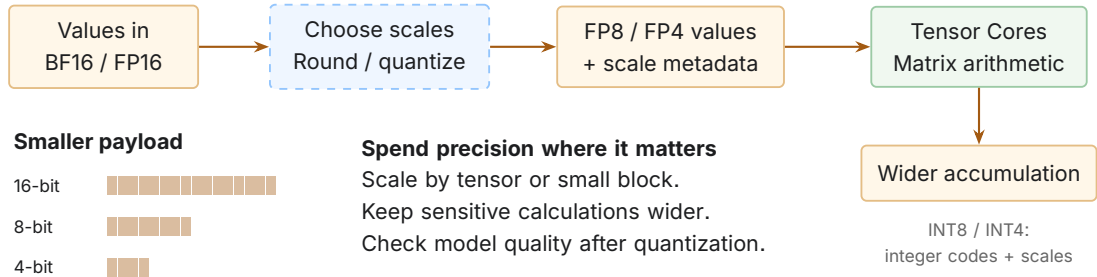
A number’s bits buy range, precision, and cost

숫자 형식은 같은 수를 얼마나 넓은 범위에서, 얼마나 촘촘하게 표현할지 정합니다. 진한 부분인 지수는 범위, 연한 부분인 가수의 소수부는 정밀도에 쓰입니다. FP16과 BF16은 둘 다 16비트지만 배분이 다릅니다. BF16은 FP32와 지수 범위를 맞추고, FP16은 소수부를 더 남깁니다.

비트를 줄이면 저장과 이동량이 줄지만 계산 오차가 달라집니다. 정밀한 과학 계산에는 FP64, 민감한 계산에는 FP32가 계속 필요합니다. 아래 TF32는 별도의 19비트 저장 형식으로 쓰는 것이 아닙니다. FP32 입력을 Tensor Core의 곱셈 앞에서 반올림하고 FP32로 누적하는 계산 모드입니다. 저장 형식과 실행 방식부터 구분해야 합니다.

Fewer bits need more numerical engineering

AN EXAMPLE LOW-PRECISION MATRIX PATH · supported hardware and kernels



The trend is mixed precision: smaller operands, selective precision, validated accuracy.

NVIDIA Transformer Engine · FP8 / FP4 and scaling · Micikevicius et al. · Mixed Precision Training (2017)

Fewer bits need more numerical engineering

최근에는 FP8, FP4, INT8, INT4처럼 숫자 하나를 더 작게 표현하는 방식이 늘었습니다. 같은 메모리에 더 많이 넣고, 더 적은 비트를 움직이며, 지원하는 행렬 회로에서 처리량을 높이려는 것입니다. 하지만 파일의 비트 수만 줄이면 끝나지는 않습니다.

그림처럼 텐서나 작은 값 묶음에 맞는 스케일을 고르고, 반올림한 값과 스케일을 함께 보관합니다. FP 계열은 부동소수점, INT 계열은 정수 코드로 값을 표현합니다. 누적과 민감한 계산은 더 넓은 정밀도를 남길 수 있습니다. 학습에서는 작은 업데이트가 사라지지 않도록 FP32 상태를 유지하기도 합니다. 모든 연산을 4비트로 바꾸는 것이 아니라, 모델과 커널에 맞는 혼합 정밀도 조합을 검증하는 방향입니다.

HBM widens the pipe; arithmetic still needs reuse

OUR STREAMING FP32 ADD · inputs read once, output written once

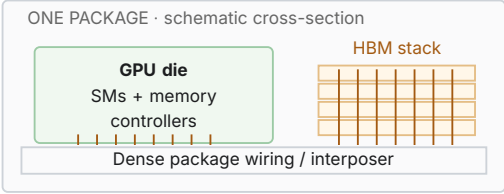


12 bytes / FLOP

An illustrative **4 TB/s** memory system:

$$4 \times 10^{12} / 12 \approx 0.33 \text{ TFLOP/s}$$

Ideal bandwidth ceiling for this add, not GPU peak compute.



Many short parallel connections.
Example: HBM3E has a 1024-bit data interface per stack.

HBM widens the pipe; arithmetic still needs reuse

앞에서 계산한 A 더하기 B로 대역폭의 의미를 보겠습니다. FP32 입력 두 개를 읽고 결과 하나를 쓰면, 덧셈 한 번에 최소 12바이트를 옮깁니다. 예를 들어 초당 4테라바이트를 공급해도, 이 조건에서 가능한 덧셈은 이상적으로 초당 약 0.33테라 번입니다. 연산 회로의 최고 성능과는 다른 상한입니다.

HBM은 메모리를 여러 층으로 쌓고 GPU 가까이 배치해, 아주 많은 병렬 배선으로 연결합니다. HBM3E는 스택 하나의 데이터 폭이 1024비트입니다. 용량이나 한 번의 접근 지연과 초당 전송량은 구분해야 합니다. 배관을 넓혀도 데이터를 계속 다시 읽으면 한계가 있으므로, 알고리즘이 가까운 저장 공간에서 값을 재사용하는 것도 중요합니다.

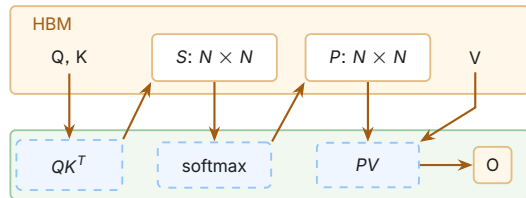
Bandwidth is bytes per second. Reusing each byte can matter as much as moving it faster.

Micron · HBM3E interface · NVIDIA · HBM package topology · A+B arithmetic: illustrative ideal bound

FlashAttention avoids writing the huge score matrix

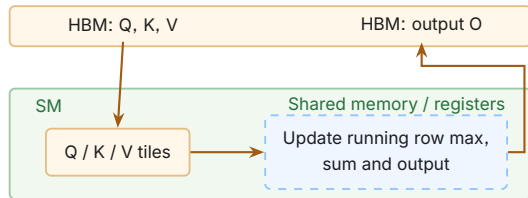
$O = \text{softmax}(QK^T / \sqrt{d})V$ · dense attention over N tokens

Separate operations



Same dense attention, less memory traffic. The arithmetic remains quadratic in sequence length.

FlashAttention · tiled and fused



FlashAttention avoids writing the huge score matrix

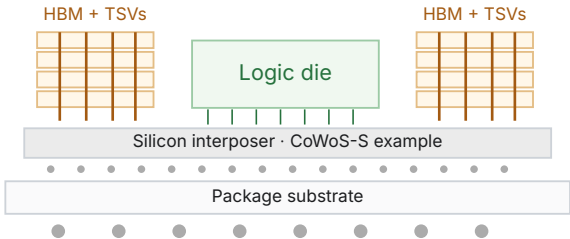
Attention을 여러 연산으로 나누면 Q와 K의 곱으로 만든 큰 점수 행렬을 HBM에 쓰고, softmax가 다시 읽은 뒤 결과를 또 씁니다. 토큰 수가 N이면 이 중간 행렬은 N 곱하기 N으로 커집니다. 계산보다 메모리를 오가는 일이 부담이 됩니다.

FlashAttention은 작은 타일만 SM의 가까운 저장 공간으로 가져옵니다. 행별 최댓값과 합, 출력의 중간값을 갱신하는 방식으로 softmax를 이어 계산해, 거대한 점수와 확률 행렬을 HBM에 통째로 만들지 않습니다. 기본 FlashAttention은 attention을 희소화하거나 근사한 방법이 아닙니다. 같은 dense attention을 다른 순서로 평가하므로 반올림 차이는 있을 수 있고, 연산량 자체가 N 제곱이라는 점도 남습니다. 하드웨어의 메모리 계층에 맞춰 알고리즘을 다시 짤 사례입니다.

Dao et al. · FlashAttention (2022), Fig. 1 and Algorithm 1 · schematic data flow; evaluation order and rounding can differ

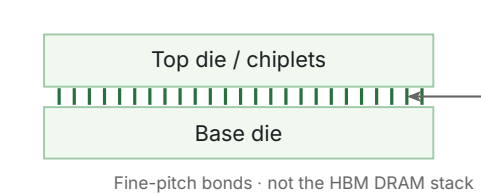
TSMC connects dies sideways and stacks them vertically

CoWoS: integrate logic + HBM in a package



CoWoS-R: RDL. CoWoS-L: local silicon links + RDL.
Schematic; sizes and layer counts are illustrative.

SoIC: dense vertical die-to-die bonds



Can be combined with CoWoS.
Constraints: tested dies, bonding yield, heat and package size.

Compute, memory, interconnect, power and cooling must work as one physical assembly.

TSMC connects dies sideways and stacks them vertically

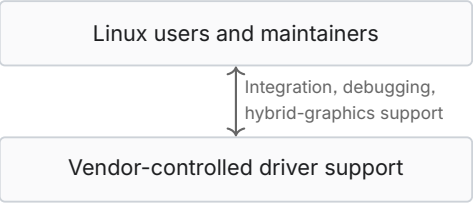
GPU와 HBM을 가깝게 놓으려면 실제로 연결하고 전원을 넣고 열도 빼야 합니다. 왼쪽 CoWoS는 계산 다이와 HBM을 하나의 패키지에 통합합니다. 그중 S 방식은 실리콘 interposer로 연결하고, R과 L은 재배선층이나 국소 실리콘 연결을 사용해 구성과 크기를 확장합니다. HBM 내부의 세로선은 메모리 층을 관통하는 TSV입니다.

오른쪽 SoIC는 별도의 다이들을 아주 촘촘한 접합으로 수직 연결하는 기술입니다. HBM의 DRAM 적층과 같은 이름으로 부르는 것은 아니며, CoWoS와 함께 사용할 수도 있습니다. 정상인 다이를 골라 조립해도 접합 수율, 발열, 전력과 패키지 크기를 해결해야 합니다. 이제 칩 성능은 트랜지스터만 작게 만드는 문제가 아니라 여러 부품을 한 시스템으로 조립하는 문제이기도 합니다.

TSMC · CoWoS-S/R/L · TSMC · SoIC · TSMC 2024 Annual Report · 3DIC thermal development

Torvalds wanted Linux to be a supported platform

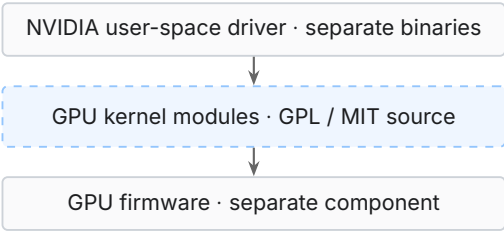
2012 · THE OPTIMUS QUESTION



The dispute concerned support and cooperation.

Fast hardware also needs maintainable operating-system integration.

2022 / 2024 · WHICH LAYER OPENED?



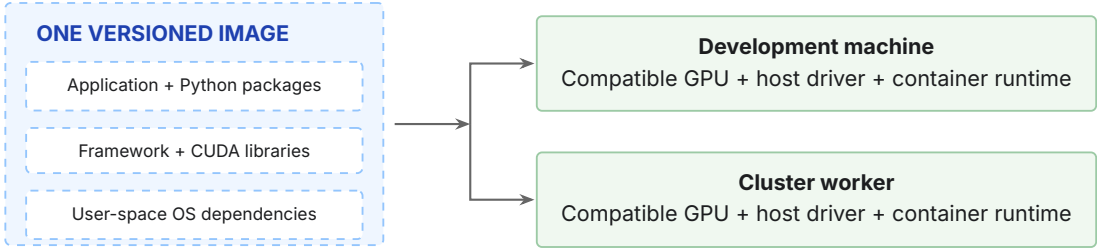
R515: source release · R560: wider default adoption

Torvalds wanted Linux to be a supported platform

유명한 장면은 2012년 Aalto 대학 대담에서 나왔습니다. 한 사용자가 노트북의 Optimus 그래픽을 Linux 에서 쓰기 어려웠던 경험을 물었고, 토발즈는 NVIDIA의 Linux 지원과 커뮤니티 협력 태도를 강하게 비판했습니다. GPU가 빠른가와 사용자가 운영체제에서 안정적으로 쓸 수 있는가는 별개의 문제였던 것입니다.

이후 상황도 구분해야 합니다. NVIDIA는 2022년 R515에서 GPU 커널 모듈의 소스를 공개했고, 2024 년 R560에서는 지원되는 GPU의 기본 선택을 이쪽으로 옮겼습니다. 오른쪽의 가운데 층이 공개된 부분입니다. 사용자 공간 드라이버와 펌웨어는 별도로 남아 있어, 전체 CUDA 스택이 공개됐다는 뜻은 아닙니다. 소스 공개와 Linux 본류에 통합되는 것도 같은 일은 아닙니다. 하드웨어를 실제 서비스에 쓰려면 이런 운영체제 경계까지 해결해야 합니다.

A GPU container carries the application environment



The host need not install each application’s CUDA Toolkit and Python environment.

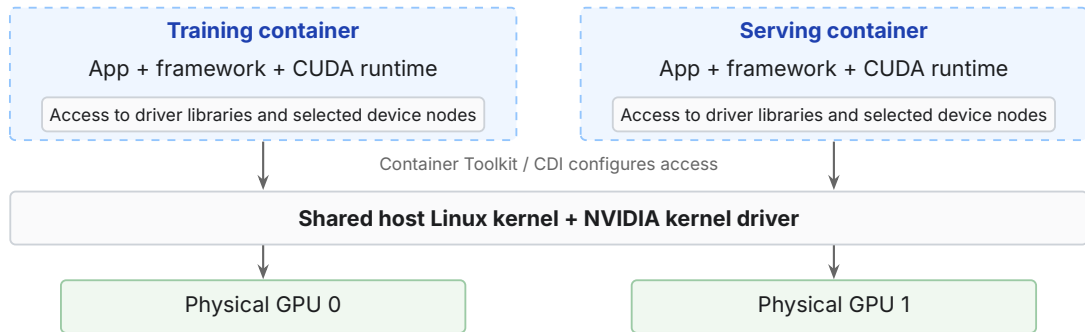
Package the user space once; deploy it on compatible GPU hosts.

A GPU container carries the application environment

GPU 컨테이너가 편리한 이유는 애플리케이션 환경을 함께 옮길 수 있기 때문입니다. Python 패키지와 프레임워크, CUDA 사용자 라이브러리를 이미지에 묶습니다. 개발 머신에서 맞춘 환경을 클러스터에서도 같은 이미지로 실행할 수 있고, 작업마다 필요한 라이브러리 버전이 달라도 호스트 전체를 그 버전에 맞춰 다시 설치할 필요가 줄어듭니다.

다만 이미지 안에 실제 GPU까지 들어가는 것은 아닙니다. 실행할 호스트에는 호환되는 GPU와 드라이버, GPU 연결을 설정한 컨테이너 실행 환경이 있어야 합니다. CPU 아키텍처와 커널에 필요한 GPU 기능도 맞아야 합니다. 환경을 재현하고 배포하는 단위가 생긴 것이 핵심이며, 모든 하드웨어에서 같은 속도나 완전히 같은 수치 결과를 보장하는 것은 아닙니다. 그러면 이미지 밖의 GPU에는 어떻게 달을까요?

The container reaches the GPU through the host driver



Containers isolate application environments; GPU access still crosses a shared driver boundary.

The container reaches the GPU through the host driver

위의 두 컨테이너는 자기 애플리케이션과 CUDA 런타임을 갖지만, 아래의 Linux 커널과 GPU 커널 드라이버는 호스트의 것을 사용합니다. Container Toolkit과 CDI 같은 연결 장치가 선택한 GPU의 장치 파일과 필요한 드라이버 라이브러리에 접근하도록 실행 환경을 구성합니다. 컨테이너가 GPU를 소프트웨어로 흉내 내는 것이 아니라 실제 장치에 작업을 제출하는 것입니다.

그래서 호스트 드라이버와 컨테이너의 CUDA 요구 조건은 여전히 맞아야 합니다. 그림에서는 GPU를 따로 배정했지만 컨테이너 개수만큼 GPU가 생기는 것은 아닙니다. 같은 GPU를 공유하면 연산과 메모리를 두고 경쟁할 수 있고, 강한 하드웨어 분할이나 사용량 관리는 별도 기능과 정책이 필요합니다. 이 경계를 연결했기 때문에 환경을 이미지로 배포하면서 실제 GPU 가속도 사용할 수 있게 된 것입니다.